



Credit Risk Analysis and Prediction Using Machine Learning

Cost-Sensitive Loan Default Prediction for Risk-Based Lending Decisions

Author: Mahdi Dadgar

Project Type: Supervised Machine Learning — Binary Classification

Domain: Financial Risk Analytics / Credit Risk Modeling

Business Use Case: Loan default prediction and risk-based loan approval support

Dataset Source: Kaggle Credit Risk Dataset

Target Variable: `loan_status`

Target Definition:

- `0` = No Default
 - `1` = Default
-

1. Project Overview

Financial institutions such as banks, fintech lenders, and credit providers face an important business challenge when issuing loans: they need to approve profitable borrowers while reducing the risk of lending to customers who may default.

This project develops a machine learning solution to predict whether a loan applicant is likely to default. The goal is not only to build a high-performing classification model, but also to evaluate the model from a business-risk perspective.

In credit risk modeling, model evaluation must go beyond accuracy. A model that looks accurate overall may still fail to detect risky borrowers. Therefore, this project focuses on recall, precision, ROC-AUC, precision-recall trade-offs, threshold selection, and business cost impact.

1.1 Business Problem

The core business problem is:

How can a financial institution identify high-risk loan applicants before approval, while avoiding unnecessary rejection of potentially good

customers?

Incorrect lending decisions can create two types of business risk:

1. **False Negative**

- The model predicts that a borrower will not default, but the borrower actually defaults.
- This is the most expensive error because the institution may approve a risky loan.

2. **False Positive**

- The model predicts that a borrower will default, but the borrower would actually repay.
- This creates lost revenue opportunity and may reduce customer acquisition.

Because false negatives are usually more costly in credit-risk settings, this project treats loan default prediction as a **cost-sensitive classification problem**.

1.2 Stakeholder and Decision Context

Primary Stakeholders

- Credit risk analysts
- Loan underwriting teams
- Risk management departments
- Financial decision-makers
- Business strategy teams

Business Decision Supported

The model is designed to support decisions such as:

- Whether to approve or reject a loan application
- Whether an application should be sent for manual review
- Whether additional risk checks are needed
- How to prioritize high-risk borrowers for further assessment

This project should be interpreted as a **decision-support prototype**, not as an automatic loan rejection system.

1.3 Machine Learning Objective

The machine learning objective is to build a binary classification model that predicts the probability of loan default using borrower, loan, and credit-history features.

The model should:

- Predict default probability for each loan applicant
 - Identify high-risk borrowers with strong recall
 - Maintain acceptable precision to avoid excessive false alarms
 - Support threshold-based business decisions
 - Provide interpretable risk drivers for stakeholders
-

1.4 Success Metrics

Because this is a credit-risk problem, the model is evaluated using multiple metrics:

Metric	Why It Matters
Recall	Measures how many actual defaulters are correctly detected
Precision	Measures how reliable default predictions are
F1-score	Balances precision and recall
ROC-AUC	Measures overall ranking/discrimination ability
Precision-Recall AUC	Useful for imbalanced classification problems
Confusion Matrix	Shows false positives and false negatives
Cost Simulation	Translates model errors into business impact

The final model will be selected based on both predictive performance and business usefulness.

1.5 Responsible AI Preview

Credit risk modeling is a high-impact application because predictions can influence access to financial services.

For that reason, this project includes responsible AI considerations:

- The model should support human decision-making, not replace it

completely.

- Model outputs should be explainable to business stakeholders.
 - High-risk predictions should be reviewed carefully before action is taken.
 - The model should be monitored for performance drift over time.
 - Potential fairness risks should be considered, especially when using personal or financial variables.
 - The dataset and model should not be used as a production lending system without further validation, compliance review, and governance.
-

1.6 Project Workflow

This notebook follows the workflow below:

1. Project introduction and business framing
 2. Environment setup
 3. Data loading and initial inspection
 4. Dataset understanding and feature interpretation
 5. Exploratory data analysis
 6. Data quality assessment
 7. Missing value treatment
 8. Outlier policy and preprocessing
 9. Feature engineering and encoding
 10. Train-test split
 11. Baseline model: Logistic Regression
 12. Threshold tuning for business alignment
 13. Advanced model: Random Forest
 14. Cross-validation and hyperparameter tuning
 15. Model comparison
 16. Precision-recall and threshold analysis
 17. Business cost simulation
 18. Model interpretation
 19. Responsible AI considerations
 20. Final deployment recommendation and business conclusion
-

1.7 Expected Project Outcome

The final version of this project will demonstrate:

- Business-oriented machine learning

- Credit-risk analytics
- Cost-sensitive classification
- Model comparison and tuning
- Threshold optimization
- Business cost simulation
- Explainable model interpretation
- Responsible AI thinking in financial decision-making

Table of Contents

- 1. Project Overview
 - 1.1 Business Problem
 - 1.2 Stakeholder and Decision Context
 - 1.3 Machine Learning Objective
 - 1.4 Success Metrics
 - 1.5 Responsible AI Preview
 - 1.6 Project Workflow
 - 1.7 Expected Project Outcome
- 2. Data Loading and Dataset Understanding
 - 2.1 Objective
 - 2.2 Dataset Context
 - 2.3 Initial Data-Quality Observations
 - 2.4 Feature Dictionary and Business Interpretation
 - 2.5 Initial Risk Hypotheses
 - 2.6 Target Variable Distribution and Class Imbalance
- 3. Exploratory Data Analysis
 - 3.1 Objective
 - 3.2 Categorical Risk Analysis
 - 3.3 Business Interpretation of Categorical Risk Patterns
 - 3.4 Numerical Feature Analysis
 - 3.5 Business Interpretation of Numerical Risk Patterns
- 4. Data Preprocessing and Data Quality Management
 - 4.1 Objective
 - 4.2 Preprocessing Principles
 - 4.3 Data Quality Issues Identified
 - 4.4 Missing Value Treatment Strategy
 - 4.5 Outlier Policy and Data Quality Corrections

- 4.6 Data Cleaning Impact Summary
- 5. Feature Engineering and Encoding
 - 5.1 Objective
 - 5.2 Modeling Preparation Strategy
 - 5.3 Preprocessing Pipeline Design
- 6. Modeling Strategy
 - 6.1 Objective
 - 6.2 Modeling Approach
 - 6.3 Evaluation Priorities
 - 6.4 Reusable Model Evaluation Function
 - 6.5 Baseline Model: Logistic Regression
 - 6.6 Cross-Validation: Logistic Regression
 - 6.7 Threshold Analysis: Logistic Regression
 - 6.8 Business Interpretation: Logistic Regression Baseline
 - 6.9 Advanced Model: Random Forest
 - 6.10 Cross-Validation: Random Forest Baseline
 - 6.11 Hyperparameter Tuning: Random Forest
 - 6.12 Model Comparison: Baseline and Tuned Models
 - 6.13 Business Interpretation of Model Comparison
 - 6.14 Random Forest Threshold Analysis
 - 6.15 Business Interpretation of Random Forest Threshold Analysis
 - 6.16 Business Cost Simulation
 - 6.17 Final Model Candidate Selection Based on Cost Simulation
- 7. Model Interpretation
 - 7.1 Objective
 - 7.2 Business Interpretation of Feature Importance
- 8. Responsible AI and Fairness Considerations
 - 8.1 Objective
 - 8.2 Why Responsible AI Matters in Credit Risk
 - 8.3 Dataset Limitations
 - 8.4 Potential Fairness Risks
 - 8.5 Responsible Use Recommendation
 - 8.6 Responsible AI Diagnostic Interpretation
- 9. Final Recommendations and Conclusion
 - 9.1 Final Model Recommendation

- 9.2 Final Model Performance
 - 9.3 Business Cost Simulation Result
 - 9.4 Recommended Business Use
 - 9.5 Key Risk Drivers
 - 9.6 Responsible AI Recommendation
 - 9.7 Limitations
 - 9.8 Future Improvements
 - 9.9 Final Conclusion
-

```
In [1]: # =====  
# Environment Setup  
# =====  
  
# Core libraries  
import os  
import warnings  
import pandas as pd  
import numpy as np  
  
# Visualization libraries  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Display settings  
from IPython.display import display  
  
# Machine learning: splitting, validation, tuning  
from sklearn.model_selection import (  
    train_test_split,  
    StratifiedKFold,  
    cross_validate,  
    GridSearchCV  
)  
  
# Machine learning: preprocessing and pipelines  
from sklearn.preprocessing import StandardScaler, OneHotEncoder  
from sklearn.compose import ColumnTransformer  
from sklearn.pipeline import Pipeline  
  
# Machine learning: models  
from sklearn.linear_model import LogisticRegression  
from sklearn.ensemble import RandomForestClassifier  
  
# Machine learning: evaluation metrics  
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,
```

```

        roc_auc_score,
        average_precision_score,
        classification_report,
        confusion_matrix,
        ConfusionMatrixDisplay,
        precision_recall_curve,
        roc_curve,
        auc
    )

    # Ignore non-critical warnings for cleaner notebook output
    warnings.filterwarnings("ignore")

    # Pandas display configuration
    pd.set_option("display.max_columns", None)
    pd.set_option("display.width", 120)

    # Plot configuration
    sns.set_theme(style="whitegrid")

    # Reproducibility settings
    RANDOM_STATE = 42
    TEST_SIZE = 0.20

    print("Environment setup completed successfully.")
    print(f"Pandas version: {pd.__version__}")
    print(f"NumPy version: {np.__version__}")

```

Environment setup completed successfully.
Pandas version: 2.2.2
NumPy version: 2.0.2

Environment Setup and Dataset Download

The dataset is downloaded directly from Kaggle using the `opendatasets` library, which simplifies authentication and ensures a reproducible workflow.

```

In [2]: # =====
# Dataset Download
# =====

# Install opendatasets quietly if needed
!pip install -q opendatasets

import opendatasets as od

# Kaggle dataset URL
DATASET_URL = "https://www.kaggle.com/datasets/laotse/credit-risk-dataset"

# Expected local dataset folder and file path
DATASET_DIR = "credit-risk-dataset"
DATA_PATH = os.path.join(DATASET_DIR, "credit_risk_dataset.csv")

```



```
# Download dataset only if it is not already available
if not os.path.exists(DATA_PATH):
    print("Dataset not found locally. Downloading from Kaggle...")
    od.download(DATASET_URL)
else:
    print("Dataset already exists locally. Skipping download.")

# Confirm dataset path
print(f"Dataset path: {DATA_PATH}")
print(f"Dataset file exists: {os.path.exists(DATA_PATH)}")
```

Dataset already exists locally. Skipping download.
Dataset path: credit-risk-dataset/credit_risk_dataset.csv
Dataset file exists: True

2. Data Loading and Dataset Understanding

2.1 Objective

Before building any machine learning model, it is important to understand the structure, meaning, quality, and business relevance of the dataset.

In this section, we inspect:

- Dataset shape
- Column names and data types
- Missing values
- Numerical feature distributions
- Categorical feature distributions
- Target variable definition
- Initial data-quality concerns

This step ensures that later preprocessing and modeling decisions are transparent, reproducible, and aligned with the credit-risk business problem.

2.2 Dataset Context

Each row in the dataset represents one loan applicant or loan application. The objective is to predict whether the applicant is likely to default on the loan.

The target variable is:

- `loan_status`
 - `0` = No Default

- 1 = Default

This makes the project a supervised binary classification problem.

From a business perspective, the model should help identify applicants with higher default risk before a loan decision is made. However, predictions should be used as decision support rather than as fully automated rejection decisions.

2.3 Initial Data-Quality Observations

The dataset contains both numerical and categorical variables. Initial inspection shows that most columns are complete, but two important variables contain missing values:

- person_emp_length
- loan_int_rate

These missing values require careful treatment because both variables may be related to credit risk:

- Employment length may reflect income stability.
- Interest rate may reflect institutional assessment of borrower risk.

The initial numerical summary also shows potential outliers or unrealistic values in variables such as age, employment length, and income. These issues will be handled later using an explicit outlier policy.

```
In [3]: # =====  
# Load Dataset  
# =====  
  
# Load the dataset using the reusable DATA_PATH variable  
df = pd.read_csv(DATA_PATH)  
  
# Create a working copy for analysis and preprocessing  
df_original = df.copy()  
  
print("Dataset loaded successfully.")  
print(f"Dataset shape: {df.shape[0]:,} rows and {df.shape[1]:,} columns")  
  
display(df.head())
```

Dataset loaded successfully.
Dataset shape: 32,581 rows and 12 columns

	person_age	person_income	person_home_ownership	person_emp_length	loan
0	22	59000	RENT	123.0	PE
1	21	9600	OWN	5.0	EDU
2	25	9600	MORTGAGE	1.0	M
3	23	65500	RENT	4.0	M
4	24	54400	RENT	8.0	M

```
In [4]: # =====
# Basic Dataset Structure
# =====

print("Dataset shape:")
print(f"Rows: {df.shape[0]:,}")
print(f"Columns: {df.shape[1]:,}")

print("\nColumn names:")
display(pd.DataFrame({
    "column_name": df.columns,
    "data_type": df.dtypes.astype(str),
    "missing_values": df.isna().sum().values,
    "missing_percentage": (df.isna().mean().values * 100).round(2),
    "unique_values": df.nunique().values
})))

print("\nNumerical feature summary:")
display(df.describe().T)

print("\nCategorical feature summary:")
display(df.describe(include="object").T)
```

Dataset shape:
Rows: 32,581
Columns: 12

Column names:

	column_name	data_type	missing_values	ratio
person_age	person_age	int64	0	0.00
person_income	person_income	int64	0	0.00
person_home_ownership	person_home_ownership	object	0	0.00
person_emp_length	person_emp_length	float64	895	2.78
loan_intent	loan_intent	object	0	0.00
loan_grade	loan_grade	object	0	0.00
loan_amnt	loan_amnt	int64	0	0.00
loan_int_rate	loan_int_rate	float64	3116	9.57
loan_status	loan_status	int64	0	0.00
loan_percent_income	loan_percent_income	float64	0	0.00
cb_person_default_on_file	cb_person_default_on_file	object	0	0.00
cb_person_cred_hist_length	cb_person_cred_hist_length	int64	0	0.00

Numerical feature summary:

	count	mean	std	min	25%
person_age	32581.0	27.734600	6.348078	20.00	23.00
person_income	32581.0	66074.848470	61983.119168	4000.00	38500.00
person_emp_length	31686.0	4.789686	4.142630	0.00	2.00
loan_amnt	32581.0	9589.371106	6322.086646	500.00	5000.00
loan_int_rate	29465.0	11.011695	3.240459	5.42	7.90
loan_status	32581.0	0.218164	0.413006	0.00	0.00
loan_percent_income	32581.0	0.170203	0.106782	0.00	0.00
cb_person_cred_hist_length	32581.0	5.804211	4.055001	2.00	3.00

Categorical feature summary:

	count	unique	top	freq
person_home_ownership	32581	4	RENT	16446
loan_intent	32581	6	EDUCATION	6453
loan_grade	32581	7	A	10777
cb_person_default_on_file	32581	2	N	26836

2.4 Feature Dictionary and Business Interpretation

To build a meaningful credit-risk model, each feature should be understood from both a technical and business perspective.

The dataset contains three main groups of variables:

1. **Borrower characteristics**
2. **Loan characteristics**
3. **Credit-history indicators**

The table below summarizes the meaning and business relevance of each variable.

Feature	Data Type	Business Meaning	Expected Relationship with Default Risk
person_age	Numerical	Age of the loan applicant	May reflect financial maturity, but extreme values require validation
person_income	Numerical	Annual income of the applicant	Lower income may increase default risk because of weaker repayment capacity
person_home_ownership	Categorical	Housing status such as rent, own, mortgage, or other	Renters may have different risk profiles than homeowners
person_emp_length	Numerical	Employment length in years	Longer employment may indicate more stable income
loan_intent	Categorical	Purpose of the loan, such as education, medical, venture, or personal	Some loan purposes may be associated with higher default risk
loan_grade	Categorical	Institutional credit grade assigned to the loan	Lower grades are expected to show higher default risk
loan_amnt	Numerical	Requested loan amount	Larger loan amounts may increase repayment burden
loan_int_rate	Numerical	Interest rate assigned to the loan	Higher interest rates often reflect higher borrower risk

Feature	Data Type	Business Meaning	Expected Relationship with Default Risk
loan_status	Binary target	Whether the borrower defaulted	Target variable for model training
loan_percent_income	Numerical	Loan amount as a percentage of income	Higher values may indicate affordability stress
cb_person_default_on_file	Categorical	Whether the borrower has a previous default on file	Previous default history is expected to increase risk
cb_person_cred_hist_length	Numerical	Length of credit history in years	Longer credit history may provide stronger evidence of borrower reliability

2.5 Initial Risk Hypotheses

Based on domain logic, the following hypotheses will be explored during EDA:

- Borrowers with higher `loan_percent_income` are expected to have higher default risk.
- Borrowers with lower `loan_grade` values are expected to default more often.
- Borrowers with a previous default on file are expected to show higher default risk.
- Borrowers with lower income may have reduced repayment capacity.
- Higher interest rates may be associated with higher institutional risk assessment.
- Some loan purposes may carry higher default rates than others.

These hypotheses will guide the exploratory analysis and later support business interpretation of the model results.

2.6 Target Variable Distribution and Class Imbalance

The target variable for this project is `loan_status`.

- `0` = No Default
- `1` = Default

Before modeling, it is important to inspect the class distribution. In credit-risk datasets, the number of non-default cases is often higher than the number of default cases. This creates a class imbalance problem.

Class imbalance matters because a model can achieve high accuracy by predicting the majority class too often, while still failing to identify risky borrowers. Therefore, this project will evaluate the model using recall, precision, F1-score, ROC-AUC, precision-recall analysis, and business cost impact rather than accuracy alone.

```
In [5]: # =====
# Target Variable Distribution
# =====

target_col = "loan_status"

# Count and percentage distribution
target_counts = df[target_col].value_counts().sort_index()
target_percentages = (df[target_col].value_counts(normalize=True).sort_index())

target_distribution = pd.DataFrame({
    "loan_status": target_counts.index,
    "class_label": ["No Default", "Default"],
    "count": target_counts.values,
    "percentage": target_percentages.values
})

display(target_distribution)

# Plot class distribution
plt.figure(figsize=(7, 5))
sns.barplot(
    data=target_distribution,
    x="class_label",
    y="count"
)

plt.title("Loan Status Distribution")
plt.xlabel("Loan Status")
plt.ylabel("Number of Applications")
plt.show()

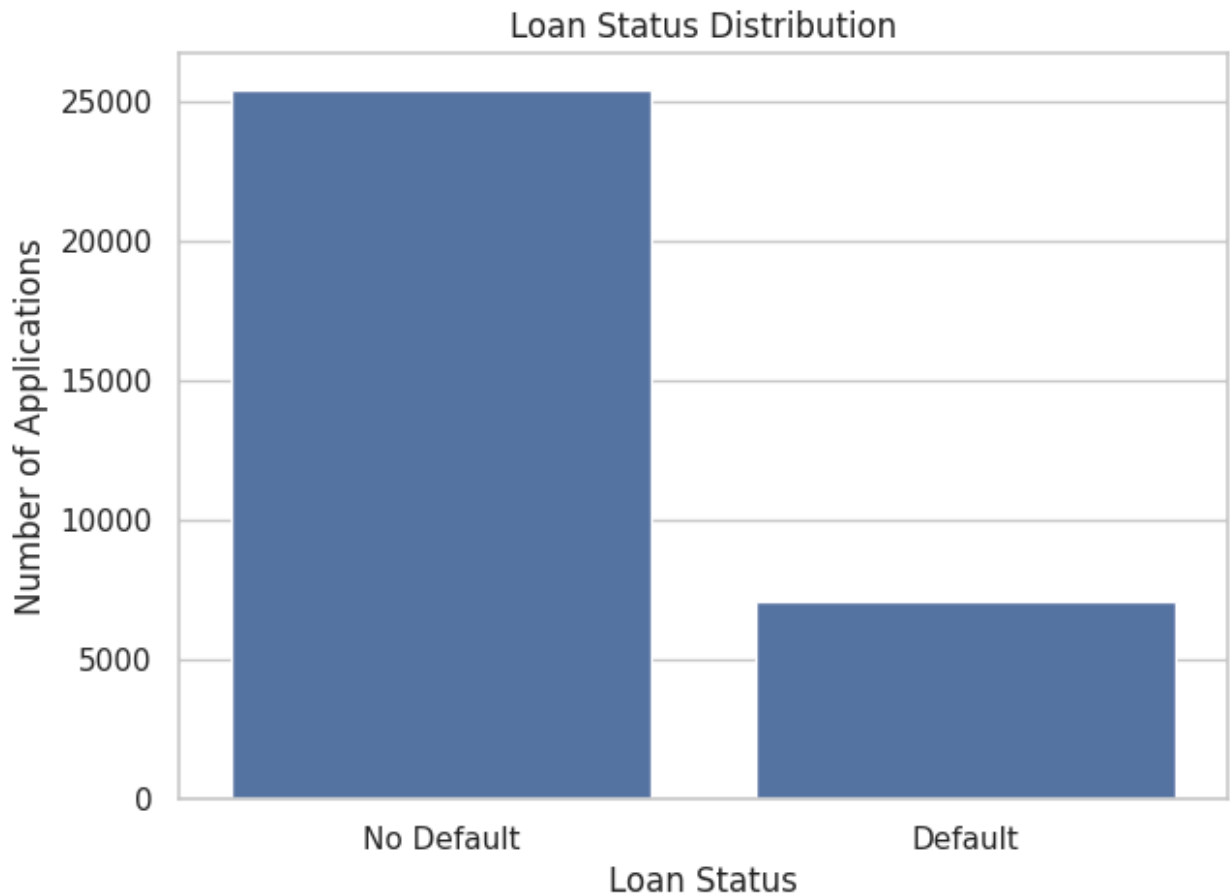
# Print business interpretation
default_rate = target_percentages.loc[1]
non_default_rate = target_percentages.loc[0]

print(f"Non-default applications: {non_default_rate:.2f}%")
print(f"Default applications: {default_rate:.2f}%")

if default_rate < 30:
    print("\nInterpretation:")
    print("The dataset is imbalanced because default cases represent a minorit
```

```
print("Accuracy alone may be misleading, so recall, precision, F1-score, F
```

	loan_status	class_label	count	percentage
0	0	No Default	25473	78.18
1	1	Default	7108	21.82



Non-default applications: 78.18%

Default applications: 21.82%

Interpretation:

The dataset is imbalanced because default cases represent a minority of the observations.

Accuracy alone may be misleading, so recall, precision, F1-score, ROC-AUC, and PR-AUC will be used for model evaluation.

3. Exploratory Data Analysis

3.1 Objective

The goal of exploratory data analysis is to understand which borrower, loan, and credit-history characteristics are associated with default risk.

In this section, we analyze:

- Default rate by categorical features
- Numerical feature distributions by loan status
- Potential risk patterns in borrower income, loan burden, interest rate, and credit history
- Data-quality issues such as extreme values and unrealistic observations
- Business insights that can guide feature engineering and model interpretation

Because the target variable is imbalanced, the EDA focuses on **default rate** rather than only raw counts. This helps identify which groups are more likely to default.

3.2 Categorical Risk Analysis

Categorical variables are important in credit-risk modeling because they often represent business-relevant borrower segments.

In this project, we analyze default rates across:

- `person_home_ownership`
- `loan_intent`
- `loan_grade`
- `cb_person_default_on_file`

These features help answer questions such as:

- Do renters default more often than homeowners?
- Are some loan purposes riskier than others?
- Does loan grade strongly separate low-risk and high-risk borrowers?
- Does previous default history increase future default risk?

The analysis below compares default rates across categorical groups and supports business interpretation before modeling.

```
In [6]: # =====  
# Default Rate by Categorical Features  
# =====  
  
categorical_features = [  
    "person_home_ownership",  
    "loan_intent",  
    "loan_grade",  
    "cb_person_default_on_file"  
]
```

```

def categorical_default_summary(data, feature, target="loan_status"):
    """
    Create a default-rate summary table for a categorical feature.
    """
    summary = (
        data.groupby(feature)
        .agg(
            applications=(target, "count"),
            defaults=(target, "sum"),
            default_rate=(target, "mean")
        )
        .reset_index()
    )

    summary["default_rate"] = (summary["default_rate"] * 100).round(2)
    summary = summary.sort_values("default_rate", ascending=False)

    return summary

categorical_summary_tables = {}

for feature in categorical_features:
    print(f"\nDefault Risk Summary by {feature}")
    summary_table = categorical_default_summary(df, feature)
    categorical_summary_tables[feature] = summary_table
    display(summary_table)

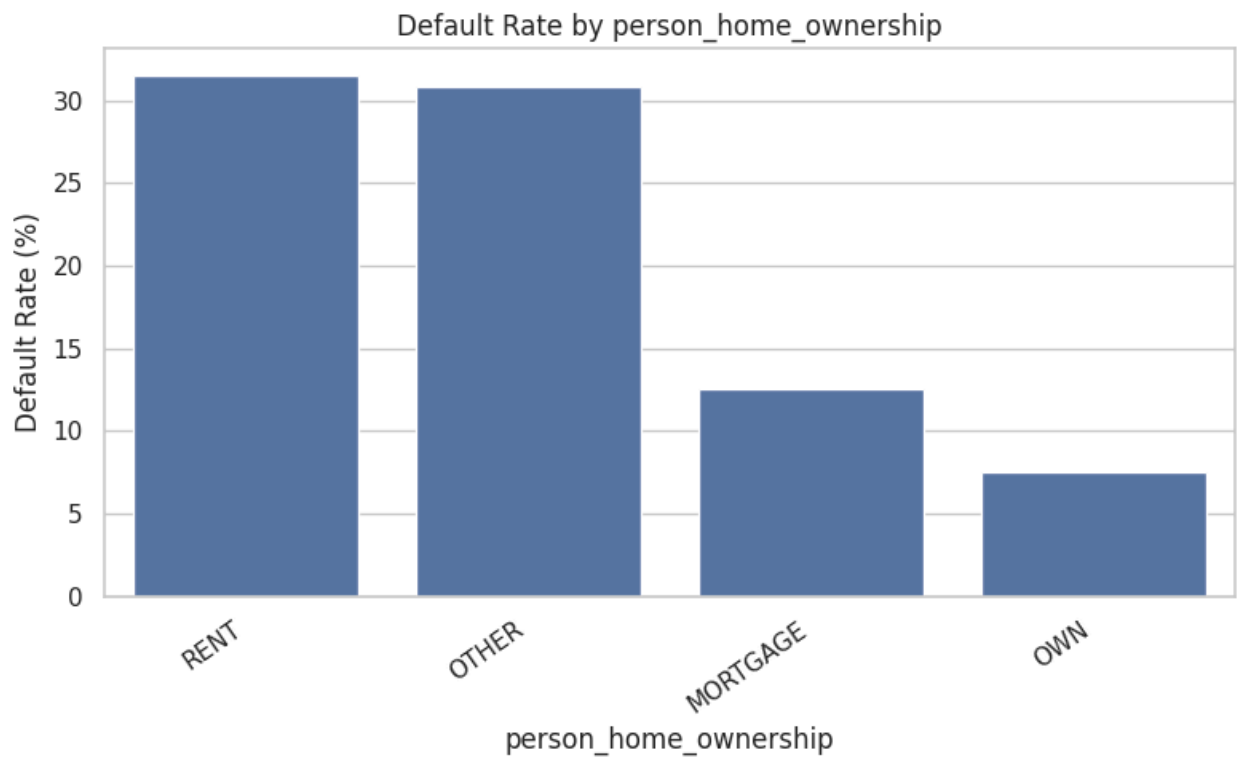
    plt.figure(figsize=(8, 5))
    sns.barplot(
        data=summary_table,
        x=feature,
        y="default_rate"
    )

    plt.title(f"Default Rate by {feature}")
    plt.xlabel(feature)
    plt.ylabel("Default Rate (%)")
    plt.xticks(rotation=35, ha="right")
    plt.tight_layout()
    plt.show()

```

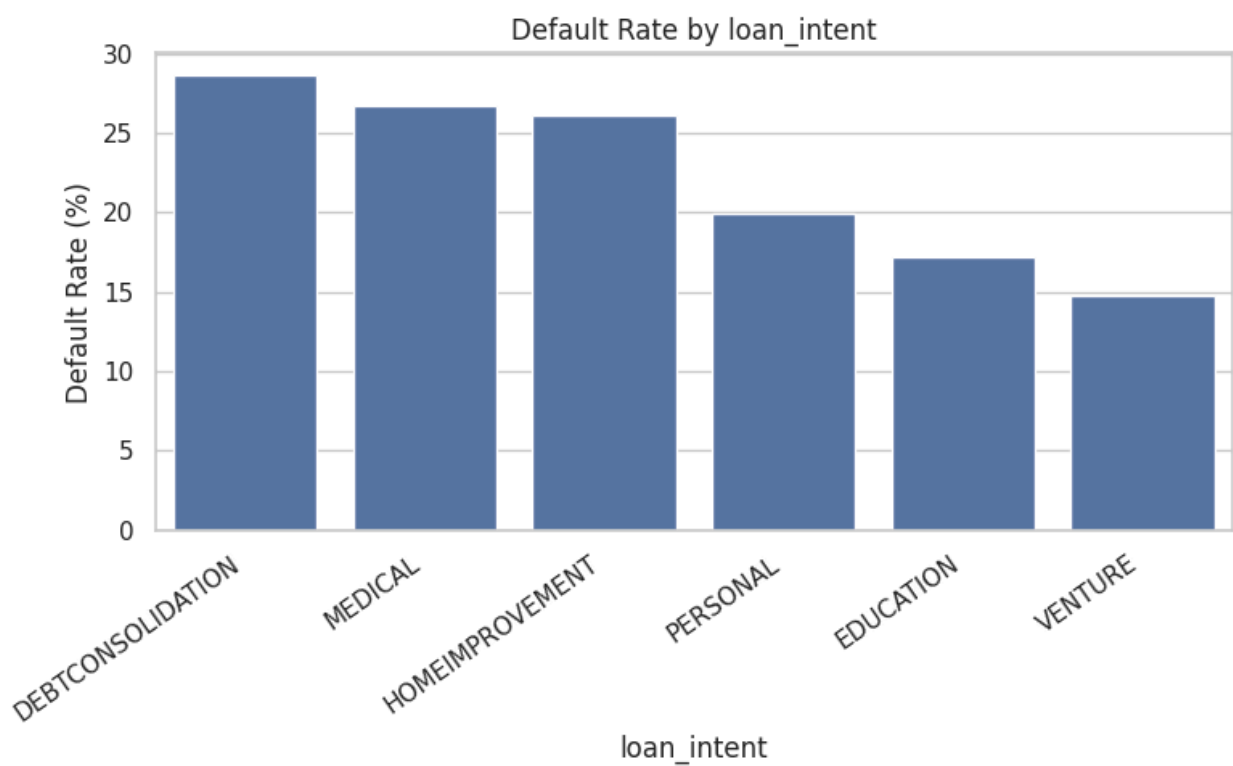
Default Risk Summary by person_home_ownership

	person_home_ownership	applications	defaults	default_rate
3	RENT	16446	5192	31.57
1	OTHER	107	33	30.84
0	MORTGAGE	13444	1690	12.57
2	OWN	2584	193	7.47



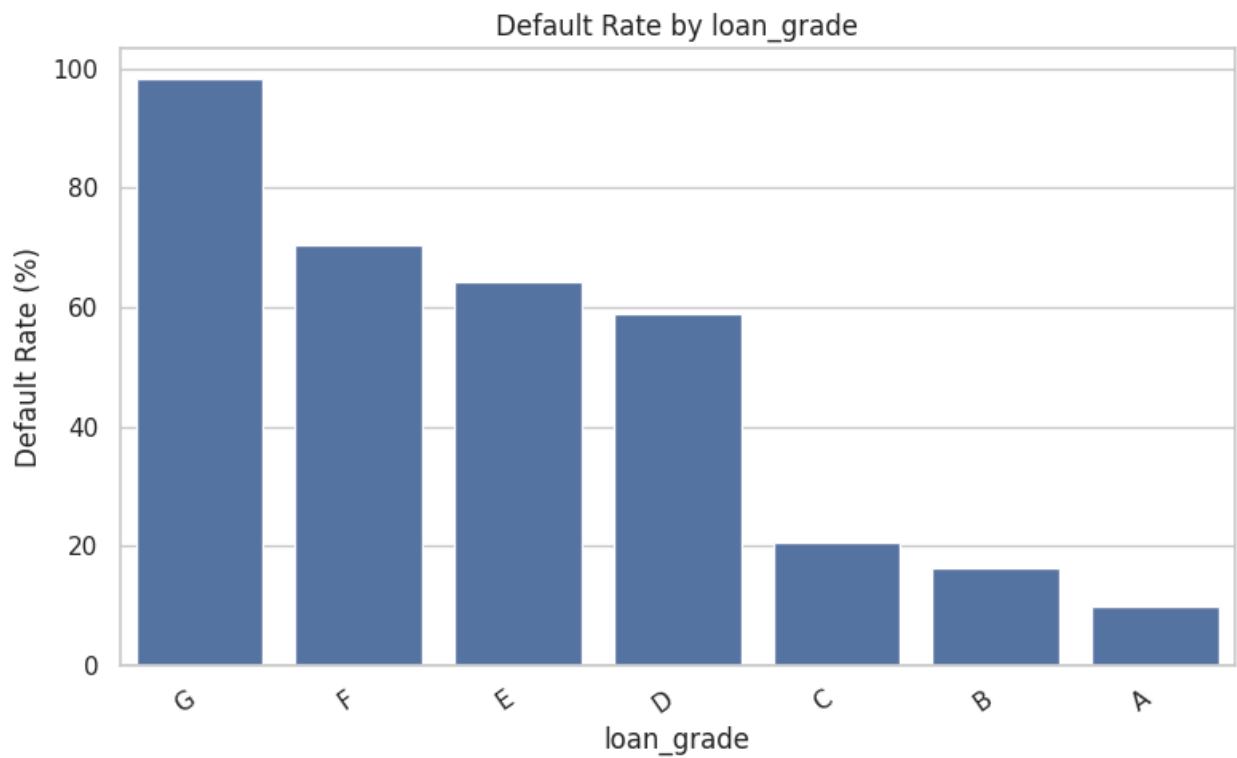
Default Risk Summary by loan_intent

	loan_intent	applications	defaults	default_rate
0	DEBTCONSOLIDATION	5212	1490	28.59
3	MEDICAL	6071	1621	26.70
2	HOMEIMPROVEMENT	3605	941	26.10
4	PERSONAL	5521	1098	19.89
1	EDUCATION	6453	1111	17.22
5	VENTURE	5719	847	14.81



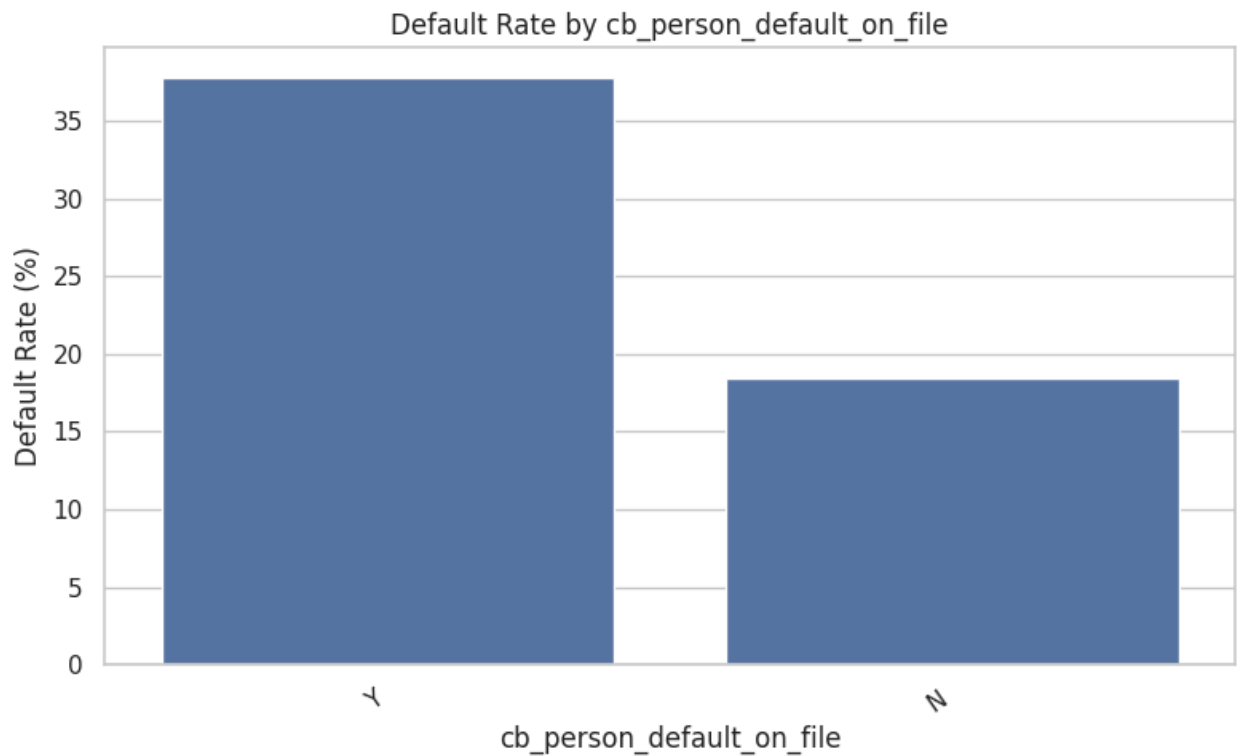
Default Risk Summary by loan_grade

	loan_grade	applications	defaults	default_rate
6	G	64	63	98.44
5	F	241	170	70.54
4	E	964	621	64.42
3	D	3626	2141	59.05
2	C	6458	1339	20.73
1	B	10451	1701	16.28
0	A	10777	1073	9.96



Default Risk Summary by cb_person_default_on_file

	cb_person_default_on_file	applications	defaults	default_rate
1	Y	5745	2172	37.81
0	N	26836	4936	18.39



3.3 Business Interpretation of Categorical Risk Patterns

The categorical default-rate analysis shows several important risk patterns that are highly relevant for credit-risk decision-making.

1. Loan Grade Is the Strongest Categorical Risk Driver

`loan_grade` shows a clear and strong relationship with default risk.

Higher-risk grades have much higher default rates:

- Grade G: 98.44%
- Grade F: 70.54%
- Grade E: 64.42%
- Grade D: 59.05%

Lower-risk grades show much lower default rates:

- Grade C: 20.73%
- Grade B: 16.28%
- Grade A: 9.96%

This pattern suggests that `loan_grade` is a powerful institutional risk signal. It likely summarizes multiple borrower-risk factors already assessed by the lender.

However, grade G has only 64 applications, so its extremely high default rate should be interpreted carefully. The overall pattern remains meaningful because the default rate generally decreases as loan grade improves.

2. Previous Default History Is a Strong Risk Indicator

Borrowers with a previous default on file show a much higher default rate than borrowers without previous default history.

- Previous default on file: 37.81%
- No previous default on file: 18.39%

This supports the business hypothesis that past credit behavior is strongly associated with future default risk.

From a lending perspective, this variable should be considered an important risk signal during underwriting and manual review.

3. Home Ownership Reflects Different Borrower Risk Profiles

Default rates differ substantially by home ownership category.

- Rent: 31.57%
- Other: 30.84%
- Mortgage: 12.57%
- Own: 7.47%

Renters show a much higher default rate than borrowers who own their home or have a mortgage. This may reflect differences in financial stability, asset ownership, or borrower profile.

However, this variable should be interpreted carefully from a responsible AI perspective. Housing status may be correlated with socioeconomic factors, so it should support risk assessment but should not be used without governance and fairness review in a real lending system.

4. Loan Intent Shows Meaningful Differences in Risk

Default rates also vary by loan purpose.

The highest-risk loan intents are:

- Debt consolidation: 28.59%
- Medical: 26.70%
- Home improvement: 26.10%

The lower-risk groups are:

- Personal: 19.89%
- Education: 17.22%
- Venture: 14.81%

This suggests that the reason for borrowing may contain useful information about repayment risk. For example, debt consolidation may indicate existing financial pressure, while medical loans may reflect unexpected financial burden.

5. Summary of Categorical EDA Insights

The categorical analysis confirms that several variables are strongly associated with loan default risk:

- `loan_grade`
- `cb_person_default_on_file`
- `person_home_ownership`
- `loan_intent`

These findings support the use of categorical variables in the machine learning model. They also provide business context for later model interpretation and feature importance analysis.

3.4 Numerical Feature Analysis

Numerical variables help explain the financial capacity, loan burden, and credit-history profile of each borrower.

In this section, we compare numerical features between borrowers who defaulted and borrowers who did not default.

The main numerical features are:

- `person_age`
- `person_income`
- `person_emp_length`
- `loan_amnt`
- `loan_int_rate`
- `loan_percent_income`
- `cb_person_cred_hist_length`

This analysis focuses on two questions:

1. Do defaulters and non-defaulters show different numerical patterns?
2. Are there unrealistic or extreme values that require cleaning before modeling?

Special attention is given to:

- High loan-to-income ratio
- Lower borrower income
- Higher interest rate
- Extreme age or employment-length values
- Extreme income observations

These patterns will guide the later data-cleaning and outlier-treatment strategy.


```

In [7]: # =====
# Numerical Features by Loan Status
# =====

numerical_features = [
    "person_age",
    "person_income",
    "person_emp_length",
    "loan_amnt",
    "loan_int_rate",
    "loan_percent_income",
    "cb_person_cred_hist_length"
]

# Summary statistics by loan status
numerical_summary_by_status = (
    df.groupby("loan_status")[numerical_features]
    .agg(["mean", "median", "std", "min", "max"])
    .T
)

display(numerical_summary_by_status)

# Business-friendly labels for plotting
df_plot = df.copy()
df_plot["loan_status_label"] = df_plot["loan_status"].map({
    0: "No Default",
    1: "Default"
})

# Boxplots for numerical features by loan status
for feature in numerical_features:
    plt.figure(figsize=(7, 4))
    sns.boxplot(
        data=df_plot,
        x="loan_status_label",
        y=feature,
        order=["No Default", "Default"]
    )
    plt.title(f"{feature} by Loan Status")
    plt.xlabel("Loan Status")
    plt.ylabel(feature)
    plt.tight_layout()
    plt.show()

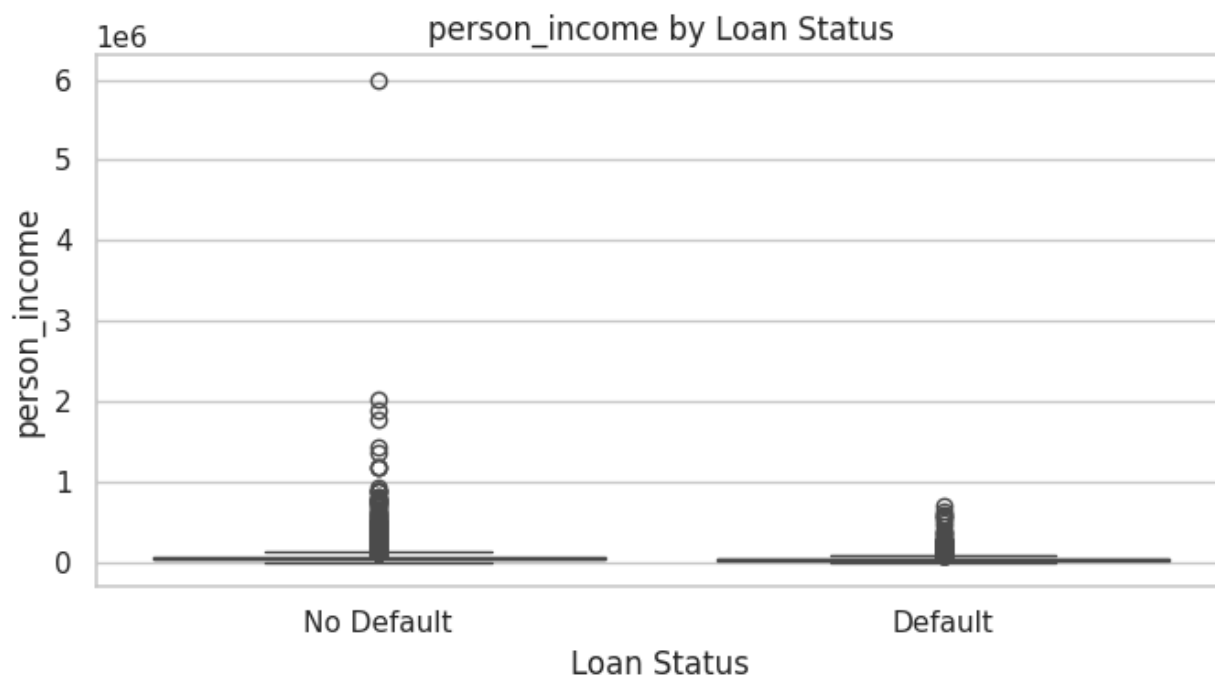
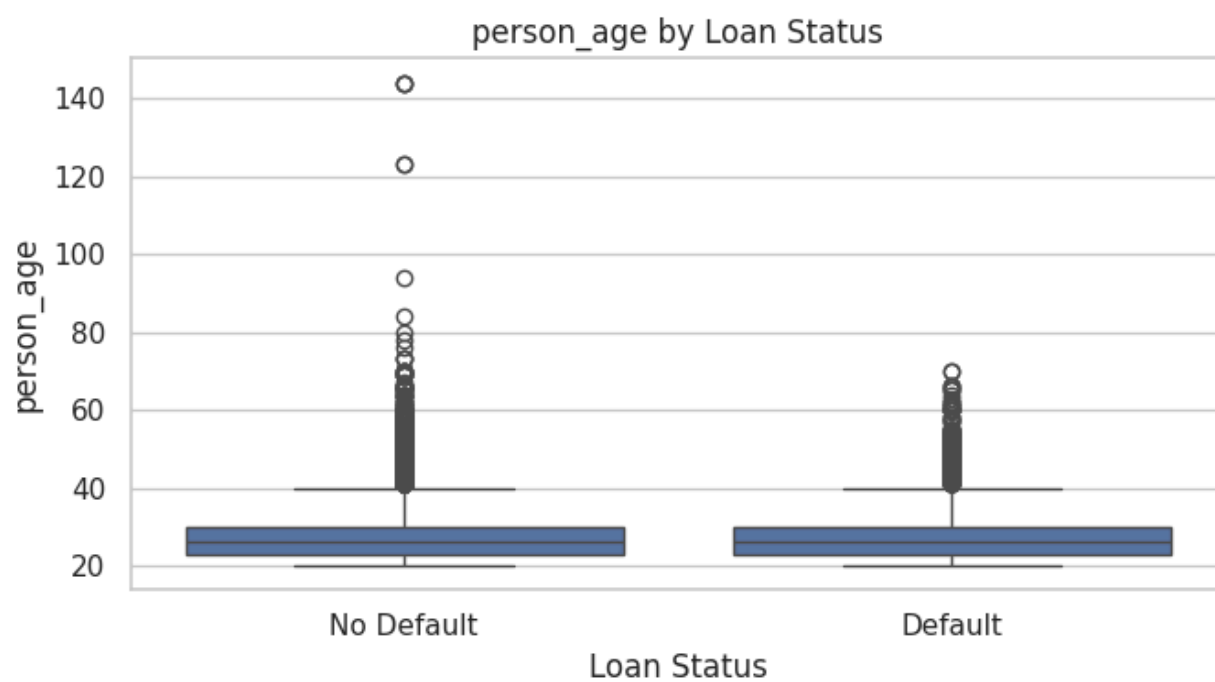
# Highlight selected business-relevant median comparisons
median_comparison = (
    df_plot.groupby("loan_status_label")[[
        "person_income",
        "loan_amnt",
        "loan_int_rate",
        "loan_percent_income"
    ]]

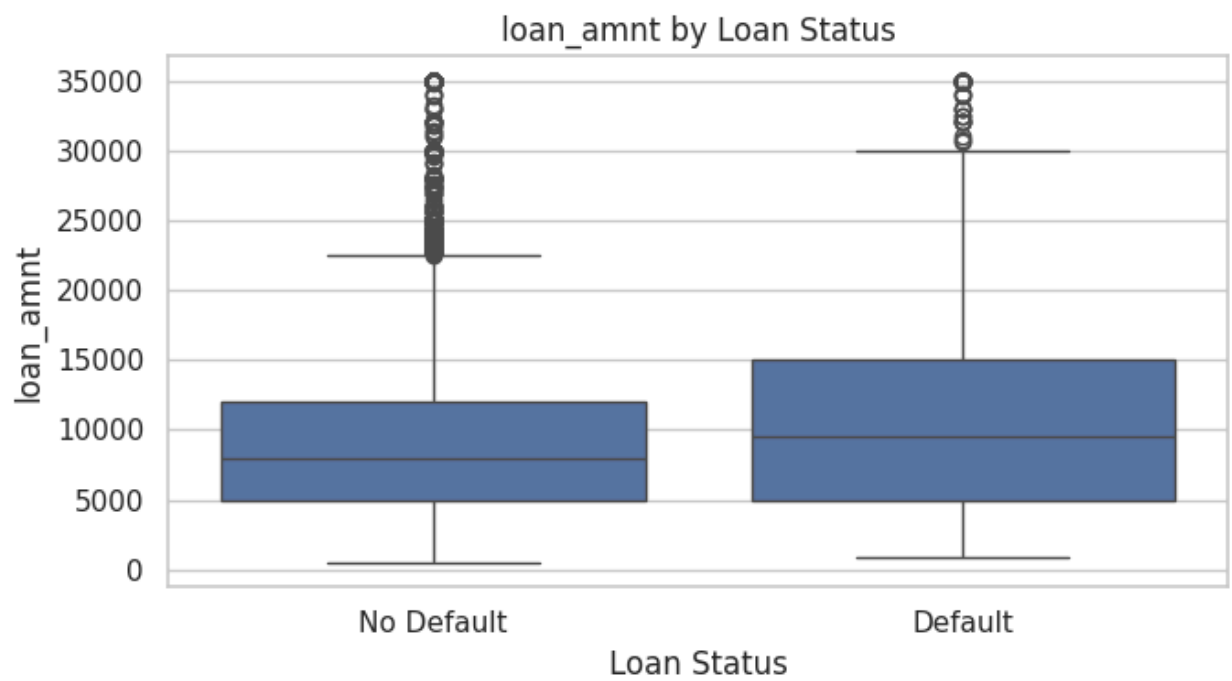
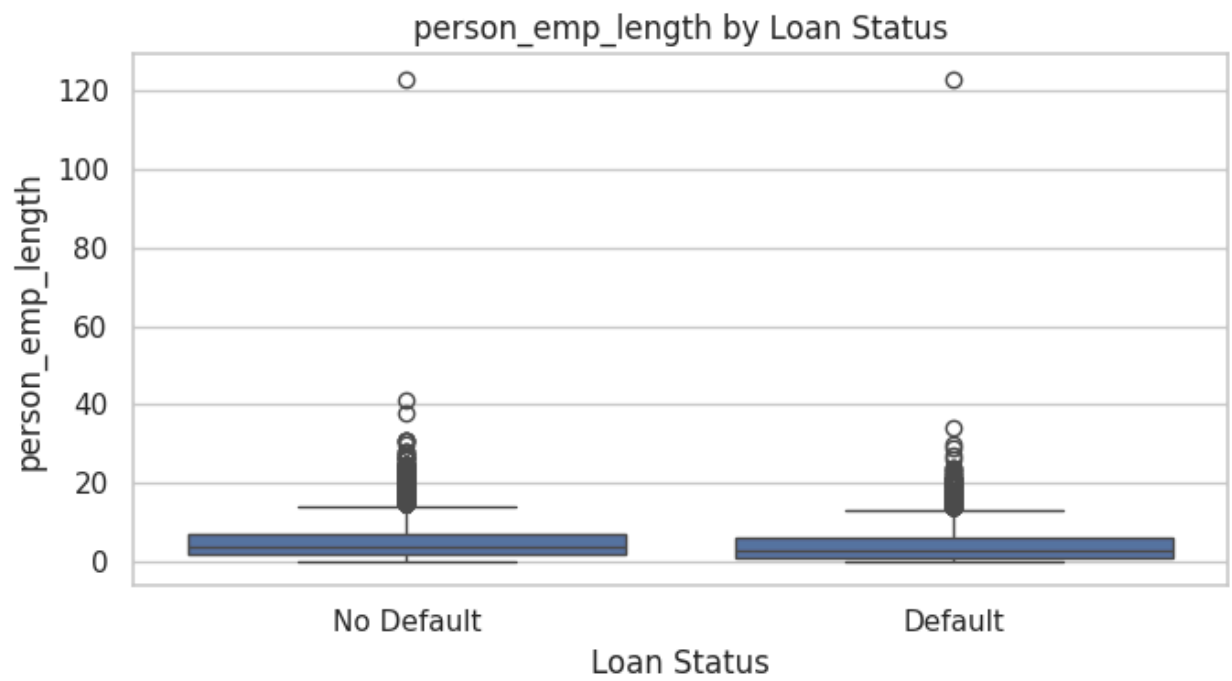
```

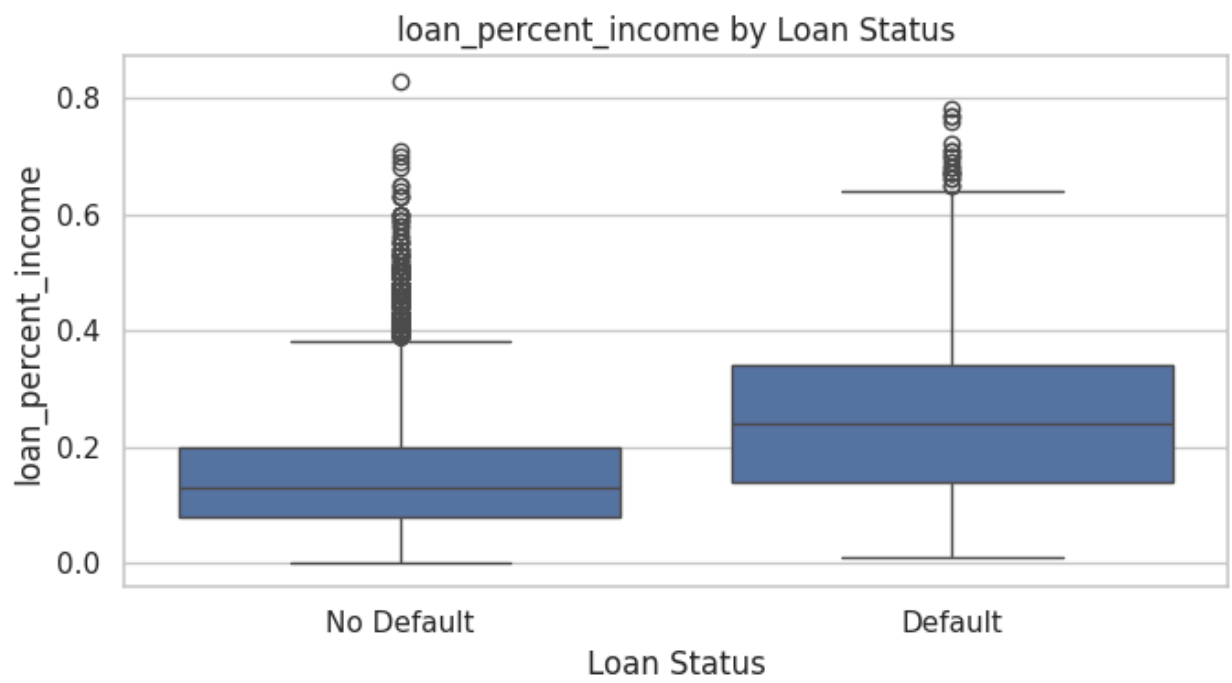
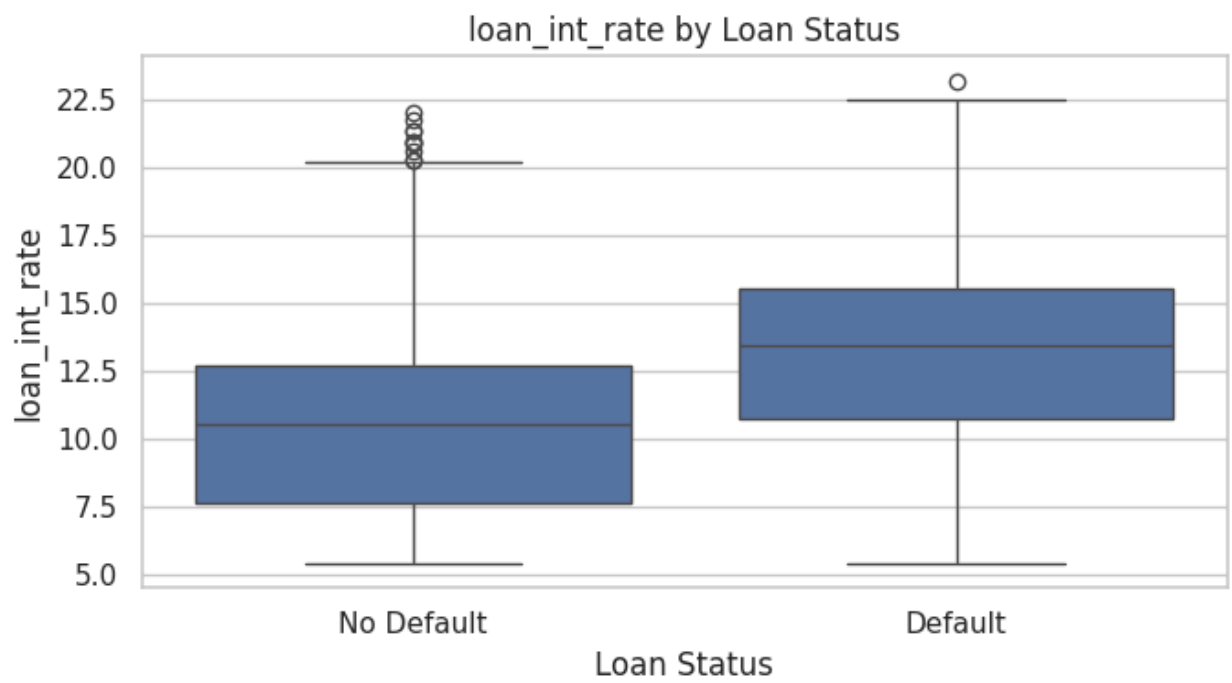
```
        .median()  
        .T  
    )  
  
    median_comparison = median_comparison[["No Default", "Default"]]  
  
    display(median_comparison)  
  
    print("Business interpretation checkpoints:")  
    print("- Compare whether defaulted borrowers have lower income than non-default")  
    print("- Compare whether defaulted borrowers have higher loan percent income.")  
    print("- Compare whether defaulted borrowers have higher interest rates.")  
    print("- Review extreme values before preprocessing, especially age, employment")
```

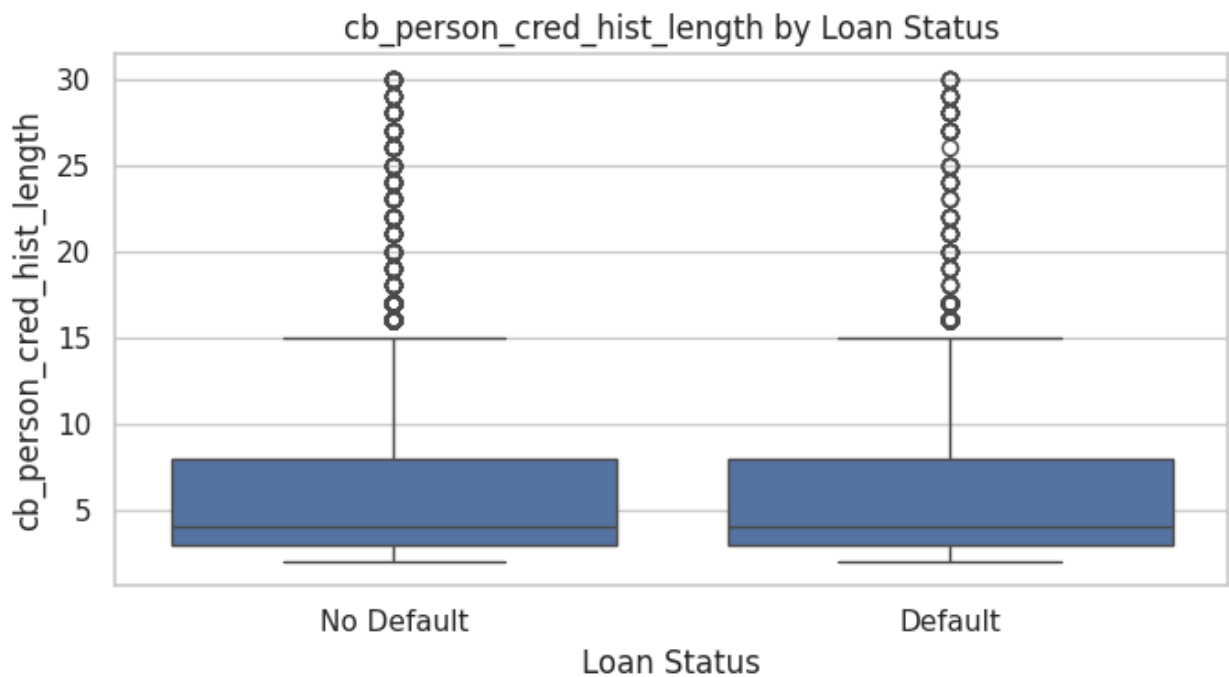
loan_status		0	1
person_age	mean	2.780713e+01	27.474676
	median	2.600000e+01	26.000000
	std	6.369669e+00	6.263642
	min	2.000000e+01	20.000000
	max	1.440000e+02	70.000000
person_income	mean	7.080436e+04	49125.652223
	median	6.000000e+04	41498.000000
	std	6.656880e+04	36907.376527
	min	7.000000e+03	4000.000000
	max	6.000000e+06	703800.000000
person_emp_length	mean	4.968745e+00	4.137562
	median	4.000000e+00	3.000000
	std	4.129022e+00	4.126956
	min	0.000000e+00	0.000000
	max	1.230000e+02	123.000000
loan_amnt	mean	9.237464e+03	10850.502954
	median	8.000000e+03	9600.000000
	std	6.026375e+03	7143.345344
	min	5.000000e+02	900.000000
	max	3.500000e+04	35000.000000
loan_int_rate	mean	1.043600e+01	13.060207
	median	1.059000e+01	13.490000
	std	2.982190e+00	3.293167
	min	5.420000e+00	5.420000
	max	2.206000e+01	23.220000
loan_percent_income	mean	1.488050e-01	0.246889
	median	1.300000e-01	0.240000
	std	8.725158e-02	0.132148
	min	0.000000e+00	0.010000
	max	8.300000e-01	0.780000
cb_person_cred_hist_length	mean	5.837475e+00	5.685003

	loan_status	0	1
	median	4.000000e+00	4.000000
	std	4.040900e+00	4.103208
	min	2.000000e+00	2.000000
	max	3.000000e+01	30.000000









loan_status_label	No Default	Default
person_income	60000.00	41498.00
loan_amnt	8000.00	9600.00
loan_int_rate	10.59	13.49
loan_percent_income	0.13	0.24

Business interpretation checkpoints:

- Compare whether defaulted borrowers have lower income than non-defaulted borrowers.
- Compare whether defaulted borrowers have higher loan percent income.
- Compare whether defaulted borrowers have higher interest rates.
- Review extreme values before preprocessing, especially age, employment length, and income.

3.5 Business Interpretation of Numerical Risk Patterns

The numerical feature analysis provides several important insights into borrower affordability, institutional risk assessment, and potential data-quality issues.

1. Defaulted Borrowers Have Lower Income

The median income of defaulted borrowers is lower than the median income of non-defaulted borrowers.

- No Default median income: 60,000
- Default median income: 41,498

This suggests that income level is an important indicator of repayment capacity. Borrowers with lower income may have less financial flexibility to manage loan repayments, especially when loan size or interest rate is high.

2. Defaulted Borrowers Have Higher Loan Burden

The median `loan_percent_income` is much higher among defaulted borrowers.

- No Default median loan percent income: 0.13
- Default median loan percent income: 0.24

This means that, for defaulted borrowers, the loan represents a larger share of income. From a credit-risk perspective, this is one of the most important affordability signals in the dataset.

A high loan-to-income ratio may indicate that the borrower is taking on debt beyond their repayment capacity.

3. Defaulted Borrowers Receive Higher Interest Rates

The median interest rate is higher for defaulted borrowers.

- No Default median interest rate: 10.59
- Default median interest rate: 13.49

This pattern suggests that interest rate may already reflect institutional risk assessment. Higher interest rates are often assigned to riskier borrowers, and the data confirms that these borrowers default more often.

This makes `loan_int_rate` an important predictive feature, but it should also be interpreted carefully because it may partially encode prior lender risk decisions.

4. Defaulted Borrowers Have Slightly Larger Loans

The median loan amount is higher for defaulted borrowers.

- No Default median loan amount: 8,000
- Default median loan amount: 9,600

Although the difference is smaller than for income, interest rate, and loan-to-income ratio, it still suggests that larger loan obligations may contribute to repayment difficulty.

Loan amount should therefore be interpreted together with income and loan

percent income rather than in isolation.

5. Extreme Values Require an Explicit Outlier Policy

The boxplots and summary statistics show several extreme or unrealistic values:

- Very high age values
- Very high employment-length values
- Extremely high income values

These values may represent data-entry errors, synthetic-data artifacts, or rare cases. If left untreated, they can distort summary statistics and influence model training.

For this reason, the preprocessing section will apply a transparent outlier policy. The goal is not to remove difficult borrowers, but to reduce the influence of unrealistic values that may harm model reliability.

6. Summary of Numerical EDA Insights

The strongest numerical risk indicators are:

- Lower borrower income
- Higher loan percent income
- Higher loan interest rate
- Slightly higher loan amount
- Potentially unrealistic outlier values requiring cleaning

These findings support the business logic of the project and will guide the preprocessing, feature engineering, and model interpretation stages.

4. Data Preprocessing and Data Quality Management

4.1 Objective

Before training machine learning models, the dataset must be cleaned and prepared in a transparent and reproducible way.

The goal of this section is to:

- Identify and document missing values
- Apply business-justified missing value treatment

- Detect unrealistic or extreme values
- Apply an explicit outlier policy
- Prepare the dataset for feature engineering and modeling
- Preserve data integrity and avoid information leakage

Data quality is especially important in credit-risk modeling because incorrect or unrealistic values can lead to misleading model results and poor lending decisions.

4.2 Preprocessing Principles

The preprocessing strategy follows these principles:

1. **Transparency**

- Every cleaning decision should be documented and explainable.

2. **Business logic**

- Cleaning rules should make sense in the credit-risk context.

3. **No target leakage**

- The target variable `loan_status` must not be used in a way that leaks information into the model.

4. **Reproducibility**

- Cleaning steps should be coded clearly so the notebook can be rerun from beginning to end.

5. **Responsible modeling**

- Data cleaning should improve reliability without removing difficult but realistic borrowers simply because they are risky.
-

4.3 Data Quality Issues Identified

Initial data inspection showed three main data-quality issues:

1. **Missing values**

- `person_emp_length`
- `loan_int_rate`

2. **Potentially unrealistic values**

- Very high age values
- Very high employment-length values
- Extremely high income values

3. Class imbalance

- Default cases represent a minority of observations.

The next cells assess these issues and apply a documented preprocessing strategy.

```
In [8]: # =====
# Missing Value Assessment
# =====

missing_summary = pd.DataFrame({
    "column_name": df.columns,
    "missing_count": df.isna().sum().values,
    "missing_percentage": (df.isna().mean().values * 100).round(2),
    "data_type": df.dtypes.astype(str).values,
    "unique_values": df.nunique().values
})

missing_summary = missing_summary.sort_values(
    by="missing_percentage",
    ascending=False
)

display(missing_summary)

# Focus only on columns with missing values
missing_only = missing_summary[missing_summary["missing_count"] > 0]

print("Columns with missing values:")
display(missing_only)

# Missing values by target class for key missing columns
missing_columns = missing_only["column_name"].tolist()

if len(missing_columns) > 0:
    for col in missing_columns:
        print(f"\nMissing rate for {col} by loan_status:")
        missing_by_target = (
            df.assign(is_missing=df[col].isna())
              .groupby("loan_status")["is_missing"]
              .mean()
              .mul(100)
              .round(2)
              .reset_index()
              .rename(columns={"is_missing": "missing_percentage"})
        )
        display(missing_by_target)
else:
```

```
print("No missing values found.")
```

	column_name	missing_count	missing_percentage	data_type	unique_value
7	loan_int_rate	3116	9.56	float64	
3	person_emp_length	895	2.75	float64	
1	person_income	0	0.00	int64	
0	person_age	0	0.00	int64	
2	person_home_ownership	0	0.00	object	
4	loan_intent	0	0.00	object	
5	loan_grade	0	0.00	object	
6	loan_amnt	0	0.00	int64	
8	loan_status	0	0.00	int64	
9	loan_percent_income	0	0.00	float64	
10	cb_person_default_on_file	0	0.00	object	
11	cb_person_cred_hist_length	0	0.00	int64	

Columns with missing values:

	column_name	missing_count	missing_percentage	data_type	unique_value
7	loan_int_rate	3116	9.56	float64	34
3	person_emp_length	895	2.75	float64	3

Missing rate for loan_int_rate by loan_status:

	loan_status	missing_percentage
0	0	9.70
1	1	9.06

Missing rate for person_emp_length by loan_status:

	loan_status	missing_percentage
0	0	2.41
1	1	3.97

4.4 Missing Value Treatment Strategy

The missing-value assessment shows that only two columns contain missing values:

- `loan_int_rate`
- `person_emp_length`

Rows with missing values will not be removed because deleting them would reduce the dataset size and may remove useful borrower information.

The treatment strategy is:

1. Create missing-value indicator features

- `loan_int_rate_missing`
- `person_emp_length_missing`

These indicators preserve information about whether a value was originally missing. This is useful because missingness itself may contain business information.

2. Impute `person_emp_length` using the median

- Employment length is a numerical variable with outliers.
- The median is more robust than the mean.

3. Impute `loan_int_rate` using the median interest rate within each `loan_grade`

- Interest rate is closely related to loan grade.
- Grade-based imputation is more business-relevant than using one global median for all borrowers.

4. Use global median as a fallback

- If any loan grade has no available median interest rate, the overall median interest rate will be used as a fallback.

This strategy preserves the dataset, limits distortion from outliers, and uses domain logic where appropriate.

```
In [9]: # =====
# Missing Value Treatment
# =====

# Create a cleaned working dataset
df_clean = df.copy()

# Preserve missingness information before imputation
df_clean["loan_int_rate_missing"] = df_clean["loan_int_rate"].isna().astype(int)
df_clean["person_emp_length_missing"] = df_clean["person_emp_length"].isna().astype(int)

# Store missing values before treatment
```

```

missing_before = df_clean.isna().sum()

# 1. Impute person_emp_length using median
emp_length_median = df_clean["person_emp_length"].median()
df_clean["person_emp_length"] = df_clean["person_emp_length"].fillna(emp_length_median)

# 2. Impute loan_int_rate using loan_grade-specific median
grade_interest_medians = df_clean.groupby("loan_grade")["loan_int_rate"].transform("median")
global_interest_median = df_clean["loan_int_rate"].median()

df_clean["loan_int_rate"] = df_clean["loan_int_rate"].fillna(grade_interest_medians)
df_clean["loan_int_rate"] = df_clean["loan_int_rate"].fillna(global_interest_median)

# Store missing values after treatment
missing_after = df_clean.isna().sum()

# Create missing-value treatment log
missing_treatment_log = pd.DataFrame({
    "missing_before": missing_before,
    "missing_after": missing_after,
    "missing_reduced": missing_before - missing_after
})

missing_treatment_log = missing_treatment_log[missing_treatment_log["missing_reduced"] > 0]

display(missing_treatment_log)

print("Missing value treatment completed.")
print(f"Employment length median used: {emp_length_median:.2f}")
print(f"Global interest rate median fallback: {global_interest_median:.2f}")
print(f"Remaining missing values in df_clean: {df_clean.isna().sum().sum()}")
print(f"Dataset shape after missing value treatment: {df_clean.shape}")

```

	missing_before	missing_after	missing_reduced
person_emp_length	895	0	895
loan_int_rate	3116	0	3116

```

Missing value treatment completed.
Employment length median used: 4.00
Global interest rate median fallback: 10.99
Remaining missing values in df_clean: 0
Dataset shape after missing value treatment: (32581, 14)

```

4.5 Outlier Policy and Data Quality Corrections

The EDA and numerical summaries showed several unrealistic or extreme values that require careful treatment before modeling.

The outlier strategy is based on business logic and data-quality principles:

1. Age

- Loan applicants should be adults.
- Very high ages such as values above 80 are treated as unrealistic for this dataset.
- Rows with `person_age > 80` are removed.

2. Employment length

- Employment length cannot realistically exceed the applicant's adult working years.
- Employment length is capped using the smaller value of:
 - 50 years
 - `person_age - 18`

3. Income

- Extremely high income values can distort model training.
- `person_income` is capped at the 99th percentile.
- This preserves most observations while reducing the influence of extreme synthetic or data-entry values.

4. Loan percent income

- Because `loan_percent_income` depends on income, it is recalculated after income capping.
- This keeps the relationship between loan amount and income internally consistent.

The goal is not to remove risky borrowers. The goal is to reduce the influence of unrealistic values and improve model reliability.

```
In [10]: # =====  
# Outlier Policy and Impact Check  
# =====  
  
# Work from the missing-value-treated dataset  
df_model = df_clean.copy()  
  
# Store before-cleaning information  
shape_before_outliers = df_model.shape  
summary_before_outliers = df_model[[  
    "person_age",  
    "person_income",  
    "person_emp_length",  
    "loan_amnt",  
    "loan_percent_income"  
]].describe().T
```

```

# -----
# 1. Remove unrealistic age values
# -----

age_filter = (df_model["person_age"] >= 18) & (df_model["person_age"] <= 80)
rows_before_age_filter = df_model.shape[0]

df_model = df_model[age_filter].copy()

rows_after_age_filter = df_model.shape[0]
age_rows_removed = rows_before_age_filter - rows_after_age_filter

# -----
# 2. Cap employment length using business logic
# -----

df_model["max_plausible_emp_length"] = (df_model["person_age"] - 18).clip(lower=

df_model["person_emp_length"] = np.minimum(
    df_model["person_emp_length"],
    df_model["max_plausible_emp_length"]
)

df_model["person_emp_length"] = np.minimum(
    df_model["person_emp_length"],
    50
)

# Remove helper column after correction
df_model = df_model.drop(columns=["max_plausible_emp_length"])

# -----
# 3. Cap extreme income values at the 99th percentile
# -----

income_cap = df_model["person_income"].quantile(0.99)

df_model["person_income"] = np.where(
    df_model["person_income"] > income_cap,
    income_cap,
    df_model["person_income"]
)

# -----
# 4. Recalculate loan_percent_income after income capping
# -----

df_model["loan_percent_income"] = df_model["loan_amnt"] / df_model["person_income"]
df_model["loan_percent_income"] = df_model["loan_percent_income"].round(4)

# Store after-cleaning information
shape_after_outliers = df_model.shape
summary_after_outliers = df_model[["

```



```

        "person_age",
        "person_income",
        "person_emp_length",
        "loan_amnt",
        "loan_percent_income"
    ]].describe().T

# -----
# 5. Create outlier treatment log
# -----

outlier_treatment_log = pd.DataFrame({
    "step": [
        "Initial dataset after missing-value treatment",
        "Remove unrealistic ages outside 18–80",
        "Cap employment length using age-based rule and 50-year maximum",
        "Cap income at 99th percentile",
        "Recalculate loan_percent_income"
    ],
    "rows_before": [
        shape_before_outliers[0],
        rows_before_age_filter,
        rows_after_age_filter,
        rows_after_age_filter,
        rows_after_age_filter
    ],
    "rows_after": [
        shape_before_outliers[0],
        rows_after_age_filter,
        rows_after_age_filter,
        rows_after_age_filter,
        rows_after_age_filter
    ],
    "rows_removed": [
        0,
        age_rows_removed,
        0,
        0,
        0
    ]
})

display(outlier_treatment_log)

print("Outlier treatment completed.")
print(f"Rows before outlier treatment: {shape_before_outliers[0]:,}")
print(f"Rows after outlier treatment: {shape_after_outliers[0]:,}")
print(f"Rows removed due to unrealistic age values: {age_rows_removed:,}")
print(f"Income cap used at 99th percentile: {income_cap:,.2f}")
print(f"Final modeling dataset shape: {df_model.shape}")

print("\nSummary before outlier treatment:")
display(summary_before_outliers)

```

```
print("\nSummary after outlier treatment:")
display(summary_after_outliers)
```

	step	rows_before	rows_after	rows_removed
0	Initial dataset after missing-value treatment	32581	32581	0
1	Remove unrealistic ages outside 18-80	32581	32574	7
2	Cap employment length using age-based rule and...	32574	32574	0
3	Cap income at 99th percentile	32574	32574	0
4	Recalculate loan_percent_income	32574	32574	0

Outlier treatment completed.
 Rows before outlier treatment: 32,581
 Rows after outlier treatment: 32,574
 Rows removed due to unrealistic age values: 7
 Income cap used at 99th percentile: 225,000.00
 Final modeling dataset shape: (32574, 14)

Summary before outlier treatment:

	count	mean	std	min	25%	50%
person_age	32581.0	27.734600	6.348078	20.0	23.00	26.00
person_income	32581.0	66074.848470	61983.119168	4000.0	38500.00	55000.00
person_emp_length	32581.0	4.767994	4.087372	0.0	2.00	4.00
loan_amnt	32581.0	9589.371106	6322.086646	500.0	5000.00	8000.00
loan_percent_income	32581.0	0.170203	0.106782	0.0	0.09	0.18

Summary after outlier treatment:

	count	mean	std	min	25%	50%
person_age	32574.0	27.714281	6.186447	20.0000	23.00	26.00
person_income	32574.0	64319.182231	37772.122268	4000.0000	38500.00	55000.00
person_emp_length	32574.0	4.290262	3.566192	0.0000	2.00	4.00
loan_amnt	32574.0	9589.199975	6322.057346	500.0000	5000.00	8000.00
loan_percent_income	32574.0	0.170772	0.106864	0.0044	0.09	0.18

4.6 Data Cleaning Impact Summary

The data preprocessing stage produced a cleaned modeling dataset called

`df_model` .

The cleaning process included:

1. Missing-value treatment
 2. Missing-value indicator creation
 3. Removal of unrealistic age values
 4. Employment-length correction
 5. Income capping at the 99th percentile
 6. Recalculation of `loan_percent_income`
-

Missing Value Treatment

Two columns contained missing values:

- `person_emp_length`
- `loan_int_rate`

Missing values were not removed. Instead, they were imputed using a business-relevant strategy:

- `person_emp_length` was filled using the median employment length.
- `loan_int_rate` was filled using loan-grade-specific median interest rates.
- Missing-value indicator columns were created to preserve information about originally missing values.

This approach keeps the dataset complete while preserving potentially useful missingness patterns.

Outlier Treatment

The outlier policy removed only 7 rows with unrealistic age values outside the 18-80 range.

This is a very small reduction compared with the original dataset size, meaning the cleaning process improved data quality without materially reducing the available training data.

Employment length was corrected using business logic so that it could not exceed a plausible working history based on age.

Income was capped at the 99th percentile to reduce the influence of extremely high values. This helps prevent unrealistic income observations from distorting

model training.

Final Cleaned Dataset

After missing-value treatment and outlier correction, the final modeling dataset contains:

- 32,574 rows
- 14 columns
- 0 remaining missing values

The additional columns are missing-value indicators:

- `loan_int_rate_missing`
- `person_emp_length_missing`

These features may help the model learn whether missing information itself is associated with credit risk.

Why This Matters for Modeling

This preprocessing strategy improves the reliability of the machine learning workflow by:

- Reducing unrealistic values
- Preserving most observations
- Avoiding unnecessary row deletion
- Maintaining business logic
- Improving reproducibility
- Creating a clear audit trail for data cleaning decisions

The cleaned dataset `df_model` will be used for all feature engineering, train-test splitting, model training, and evaluation steps from this point onward.

5. Feature Engineering and Encoding

5.1 Objective

The cleaned dataset `df_model` will now be prepared for machine learning.

This section separates the target variable from the input features and identifies which columns require numerical scaling or categorical encoding.

The modeling dataset includes:

- Original borrower characteristics
- Original loan characteristics
- Original credit-history variables
- Missing-value indicator features created during preprocessing
- Cleaned and corrected numerical variables

The target variable is:

- `loan_status`

The feature matrix will be called:

- `X`

The target vector will be called:

- `y`

5.2 Modeling Preparation Strategy

The modeling strategy is designed to avoid data leakage and support reproducible machine learning.

The main preparation steps are:

1. Separate features and target.
2. Identify categorical and numerical columns.
3. Apply one-hot encoding to categorical variables.
4. Apply scaling to numerical variables where needed.
5. Build preprocessing pipelines that can be used consistently across models.

Tree-based models such as Random Forest do not require feature scaling, but Logistic Regression benefits from scaled numerical features. To keep the workflow consistent and professional, preprocessing will be handled using scikit-learn pipelines.

```
In [11]: # =====  
# Feature and Target Separation  
# =====  
  
# Define target column  
target_col = "loan_status"
```

```

# Separate features and target using the cleaned modeling dataset
X = df_model.drop(columns=[target_col])
y = df_model[target_col]

# Identify categorical and numerical features
categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()
numerical_cols = X.select_dtypes(include=["int64", "float64"]).columns.tolist()

# Create a feature overview table
feature_overview = pd.DataFrame({
    "feature_name": X.columns,
    "data_type": X.dtypes.astype(str).values,
    "feature_group": [
        "Categorical" if col in categorical_cols else "Numerical"
        for col in X.columns
    ],
    "missing_values": X.isna().sum().values,
    "unique_values": X.nunique().values
})

display(feature_overview)

print(f"Feature matrix shape: {X.shape}")
print(f"Target vector shape: {y.shape}")
print(f"Number of numerical features: {len(numerical_cols)}")
print(f"Number of categorical features: {len(categorical_cols)}")

print("\nNumerical features:")
print(numerical_cols)

print("\nCategorical features:")
print(categorical_cols)

```

	feature_name	data_type	feature_group	missing_values	unique_v
0	person_age	int64	Numerical	0	
1	person_income	float64	Numerical	0	
2	person_home_ownership	object	Categorical	0	
3	person_emp_length	float64	Numerical	0	
4	loan_intent	object	Categorical	0	
5	loan_grade	object	Categorical	0	
6	loan_amnt	int64	Numerical	0	
7	loan_int_rate	float64	Numerical	0	
8	loan_percent_income	float64	Numerical	0	
9	cb_person_default_on_file	object	Categorical	0	
10	cb_person_cred_hist_length	int64	Numerical	0	
11	loan_int_rate_missing	int64	Numerical	0	
12	person_emp_length_missing	int64	Numerical	0	

Feature matrix shape: (32574, 13)

Target vector shape: (32574,)

Number of numerical features: 9

Number of categorical features: 4

Numerical features:

```
['person_age', 'person_income', 'person_emp_length', 'loan_amnt', 'loan_int_rate', 'loan_percent_income', 'cb_person_cred_hist_length', 'loan_int_rate_missing', 'person_emp_length_missing']
```

Categorical features:

```
['person_home_ownership', 'loan_intent', 'loan_grade', 'cb_person_default_on_file']
```

5.3 Preprocessing Pipeline Design

The dataset contains both numerical and categorical variables, so preprocessing must be handled carefully before modeling.

Two preprocessing pipelines are defined:

1. Linear-model preprocessing

- Numerical features are scaled using `StandardScaler`.
- Categorical features are encoded using one-hot encoding.
- This pipeline is appropriate for Logistic Regression.

2. Tree-model preprocessing

- Numerical features are passed through without scaling.
- Categorical features are encoded using one-hot encoding.
- This pipeline is appropriate for Random Forest and other tree-based models.

The preprocessing objects are defined at this stage but not fitted on the full dataset. They will be fitted later inside model pipelines after the train-test split. This avoids data leakage and keeps the workflow reproducible.

```
In [12]: # =====  
# Preprocessing Pipeline Definitions  
# =====  
  
def make_onehot_encoder():  
    """  
    Create a OneHotEncoder with compatibility across scikit-learn versions.  
    Newer versions use sparse_output=False, older versions use sparse=False.  
    """  
    try:  
        return OneHotEncoder(handle_unknown="ignore", sparse_output=False)  
    except TypeError:  
        return OneHotEncoder(handle_unknown="ignore", sparse=False)  
  
# Preprocessing for Logistic Regression  
# Numerical features are scaled; categorical features are one-hot encoded.  
preprocessor_linear = ColumnTransformer(  
    transformers=[  
        ("num", StandardScaler(), numerical_cols),  
        ("cat", make_onehot_encoder(), categorical_cols)  
    ],  
    remainder="drop"  
)  
  
# Preprocessing for Random Forest  
# Numerical features are not scaled; categorical features are one-hot encoded.  
preprocessor_tree = ColumnTransformer(  
    transformers=[  
        ("num", "passthrough", numerical_cols),  
        ("cat", make_onehot_encoder(), categorical_cols)  
    ],  
    remainder="drop"  
)  
  
# Summarize categorical cardinality and expected encoded feature count  
categorical_cardinality = pd.DataFrame({
```



```

    "categorical_feature": categorical_cols,
    "unique_categories": [X[col].nunique() for col in categorical_cols]
})

display(categorical_cardinality)

estimated_encoded_feature_count = len(numerical_cols) + categorical_cardinality

preprocessing_summary = pd.DataFrame({
    "pipeline": ["preprocessor_linear", "preprocessor_tree"],
    "intended_model_type": ["Logistic Regression", "Random Forest / Tree-Based Models"],
    "numerical_treatment": ["StandardScaler", "Passthrough"],
    "categorical_treatment": ["OneHotEncoder", "OneHotEncoder"],
    "number_of_numerical_features": [len(numerical_cols), len(numerical_cols)],
    "number_of_categorical_features": [len(categorical_cols), len(categorical_cols)],
    "estimated_features_after_encoding": [
        estimated_encoded_feature_count,
        estimated_encoded_feature_count
    ]
})

display(preprocessing_summary)

print("Preprocessing pipelines defined successfully.")
print(f"Estimated number of features after one-hot encoding: {estimated_encoded_feature_count}")
print("Important: preprocessors are defined but not fitted yet. They will be fitted in the")

```

	categorical_feature	unique_categories
0	person_home_ownership	4
1	loan_intent	6
2	loan_grade	7
3	cb_person_default_on_file	2

	pipeline	intended_model_type	numerical_treatment	categorical_treatment
0	preprocessor_linear	Logistic Regression	StandardScaler	OneHotEncoder
1	preprocessor_tree	Random Forest / Tree-Based Models	Passthrough	OneHotEncoder

Preprocessing pipelines defined successfully.
Estimated number of features after one-hot encoding: 28
Important: preprocessors are defined but not fitted yet. They will be fitted in the side model pipelines after train-test split.

6. Modeling Strategy

6.1 Objective

The objective of the modeling stage is to build, evaluate, and compare machine learning models that predict loan default risk.

Because this is a credit-risk classification problem, model selection should not be based on accuracy alone. The dataset is imbalanced, and the business cost of different error types is not equal.

The modeling strategy focuses on:

- Detecting default cases effectively
 - Reducing false negatives
 - Maintaining acceptable precision
 - Comparing baseline and advanced models
 - Evaluating threshold trade-offs
 - Translating model errors into business impact
-

6.2 Modeling Approach

This project uses the following modeling workflow:

1. **Stratified train-test split**
 - Preserves the same default/non-default class distribution in training and test sets.
2. **Baseline model**
 - Logistic Regression is used as an interpretable baseline model.
3. **Advanced model**
 - Random Forest is used to capture non-linear patterns and feature interactions.
4. **Cross-validation**
 - Used to evaluate model stability on the training data.
5. **Hyperparameter tuning**
 - Used to improve Random Forest performance.
6. **Threshold analysis**

- Used to select a business-aligned decision threshold instead of relying only on the default 0.50 threshold.

7. Business cost simulation

- Used to compare the financial impact of false positives and false negatives.

6.3 Evaluation Priorities

The key evaluation metrics are:

Metric	Purpose
Recall	Measures how many actual defaulters are detected
Precision	Measures how reliable default predictions are
F1-score	Balances precision and recall
ROC-AUC	Measures ranking ability across thresholds
PR-AUC	Useful for imbalanced classification
Confusion Matrix	Shows false positives and false negatives
Cost Simulation	Connects model errors to business impact

In this project, recall for the default class is especially important because missed defaulters can create direct financial losses.

```
In [13]: # =====
# Stratified Train-Test Split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=y
)

print("Train-test split completed successfully.")
print(f"Training feature matrix shape: {X_train.shape}")
print(f"Test feature matrix shape: {X_test.shape}")
print(f"Training target shape: {y_train.shape}")
print(f"Test target shape: {y_test.shape}")

# Compare target distribution in full dataset, training set, and test set
split_distribution = pd.DataFrame({
    "Full Dataset": y.value_counts(normalize=True).sort_index(),
```

```

    "Training Set": y_train.value_counts(normalize=True).sort_index(),
    "Test Set": y_test.value_counts(normalize=True).sort_index()
})

split_distribution = (split_distribution * 100).round(2)
split_distribution.index = ["No Default", "Default"]

display(split_distribution)

print("Class distribution check:")
print("The train and test sets preserve the original class distribution through stratified splitting.")

```

Train-test split completed successfully.
 Training feature matrix shape: (26059, 13)
 Test feature matrix shape: (6515, 13)
 Training target shape: (26059,)
 Test target shape: (6515,)

	Full Dataset	Training Set	Test Set
No Default	78.18	78.18	78.17
Default	21.82	21.82	21.83

Class distribution check:
 The train and test sets preserve the original class distribution through stratified splitting.

6.4 Reusable Model Evaluation Function

Before training models, a reusable evaluation function is created to ensure that all models are assessed using the same metrics.

The function evaluates:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- PR-AUC
- Confusion matrix values
- Classification report

This is important because the project compares multiple models and later tests different decision thresholds.

Using one evaluation function makes the notebook more consistent, reproducible, and easier to interpret.

```

In [14]: # =====
# Reusable Model Evaluation Function
# =====

def evaluate_classification_model(
    model,
    X_data,
    y_true,
    model_name="Model",
    threshold=0.50,
    show_confusion_matrix=True
):
    """
    Evaluate a binary classification model using probability predictions.

    Parameters:
    - model: fitted scikit-learn pipeline or classifier
    - X_data: feature matrix
    - y_true: true labels
    - model_name: name used in output tables
    - threshold: decision threshold for converting probabilities into class labels
    - show_confusion_matrix: whether to display a confusion matrix plot

    Returns:
    - metrics_dict: dictionary with key evaluation metrics
    """

    # Predicted probabilities for the positive class
    y_proba = model.predict_proba(X_data)[: , 1]

    # Convert probabilities into class predictions using selected threshold
    y_pred = (y_proba >= threshold).astype(int)

    # Confusion matrix values
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

    # Evaluation metrics
    metrics_dict = {
        "model": model_name,
        "threshold": threshold,
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1_score": f1_score(y_true, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_true, y_proba),
        "pr_auc": average_precision_score(y_true, y_proba),
        "true_negatives": tn,
        "false_positives": fp,
        "false_negatives": fn,
        "true_positives": tp
    }

    # Display metrics table

```

```

metrics_table = pd.DataFrame([metrics_dict])
display(metrics_table)

# Display classification report
print(f"Classification Report: {model_name}")
print(classification_report(
    y_true,
    y_pred,
    target_names=["No Default", "Default"],
    zero_division=0
))

# Display confusion matrix
if show_confusion_matrix:
    disp = ConfusionMatrixDisplay(
        confusion_matrix=confusion_matrix(y_true, y_pred),
        display_labels=["No Default", "Default"]
    )
    disp.plot(values_format="d")
    plt.title(f"Confusion Matrix – {model_name} | Threshold = {threshold}")
    plt.show()

return metrics_dict

print("Reusable evaluation function created successfully.")

```

Reusable evaluation function created successfully.

6.5 Baseline Model: Logistic Regression

Logistic Regression is used as the baseline model because it is simple, interpretable, and commonly used for binary classification problems.

In this project, Logistic Regression provides a reference point for comparing more advanced models such as Random Forest.

The model is built inside a scikit-learn pipeline that includes:

- Numerical feature scaling using `StandardScaler`
- One-hot encoding for categorical variables
- Logistic Regression classifier

Because the dataset is imbalanced, `class_weight="balanced"` is used to give more weight to the minority default class. This helps the model pay more attention to default cases.

The model is first evaluated using the default classification threshold of 0.50. Later, threshold tuning will be used to explore whether a lower threshold improves business performance.

```
In [15]: # =====
# Baseline Model: Logistic Regression
# =====

# Create Logistic Regression pipeline
log_reg_model = Pipeline(steps=[
    ("preprocessor", preprocessor_linear),
    ("model", LogisticRegression(
        max_iter=1000,
        class_weight="balanced",
        random_state=RANDOM_STATE
    ))
])

# Train model
log_reg_model.fit(X_train, y_train)

print("Logistic Regression model trained successfully.")

# Evaluate model on test set using default threshold
model_results = []

log_reg_metrics = evaluate_classification_model(
    model=log_reg_model,
    X_data=X_test,
    y_true=y_test,
    model_name="Logistic Regression Baseline",
    threshold=0.50
)

model_results.append(log_reg_metrics)
```

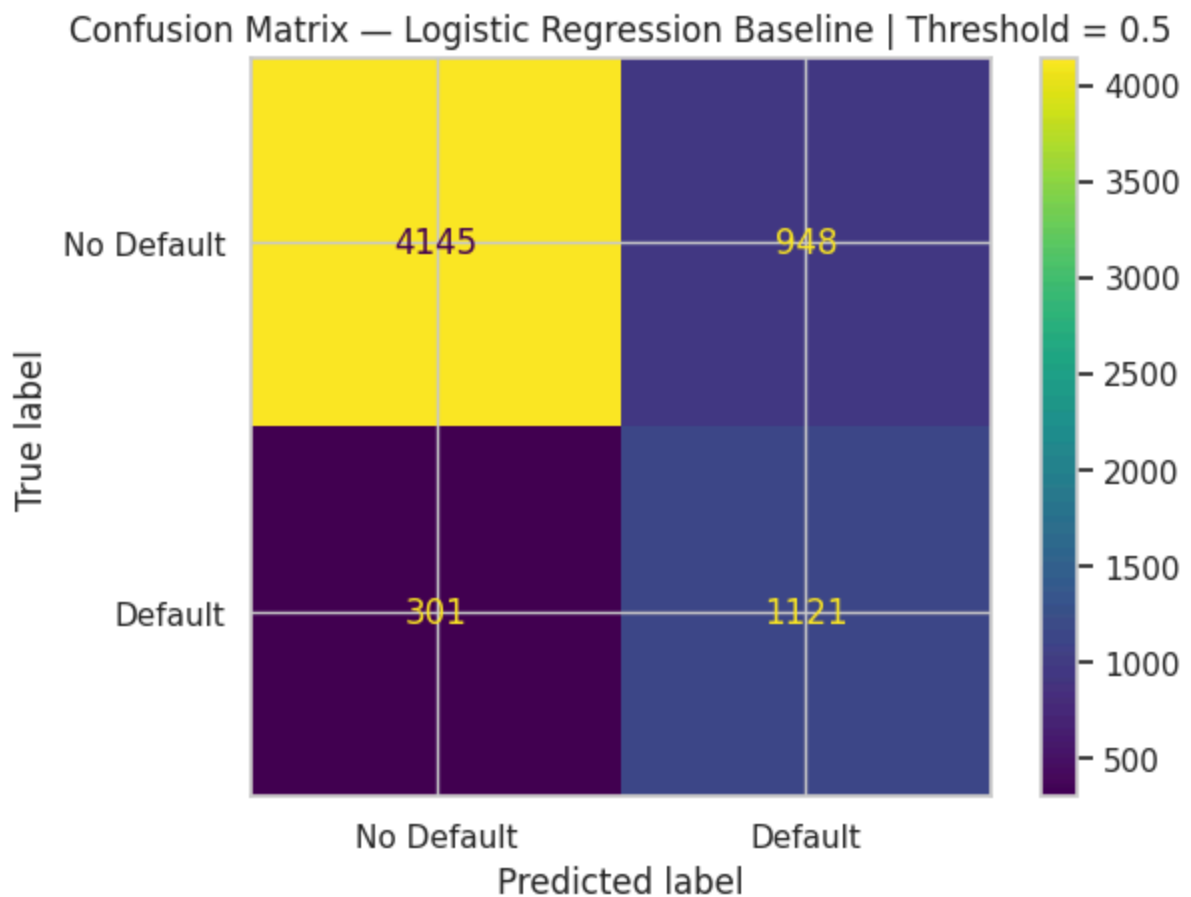
Logistic Regression model trained successfully.

	model	threshold	accuracy	precision	recall	f1_score	roc_auc	pr_auc
0	Logistic Regression Baseline	0.5	0.808289	0.541808	0.788326	0.642223	0.87101	0.71645

Classification Report: Logistic Regression Baseline

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

No Default	0.93	0.81	0.87	5093
Default	0.54	0.79	0.64	1422
accuracy			0.81	6515
macro avg	0.74	0.80	0.76	6515
weighted avg	0.85	0.81	0.82	6515



6.6 Cross-Validation: Logistic Regression

The Logistic Regression baseline was first evaluated on the holdout test set. To better understand model stability, cross-validation is also applied on the training data.

Cross-validation helps answer the question:

Does the model perform consistently across different training folds, or is the performance highly dependent on one specific train-test split?

A 5-fold stratified cross-validation strategy is used. Stratification preserves the class distribution in each fold, which is important because the dataset is imbalanced.

The model is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

- ROC-AUC
- PR-AUC

For this credit-risk problem, recall and PR-AUC are especially important because the default class is the minority class and missed defaulters can create financial losses.

```
In [16]: # =====
# Cross-Validation: Logistic Regression
# =====

# Define stratified cross-validation strategy
cv_strategy = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE
)

# Define scoring metrics
scoring_metrics = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1_score": "f1",
    "roc_auc": "roc_auc",
    "pr_auc": "average_precision"
}

# Run cross-validation on the training data
log_reg_cv_results = cross_validate(
    estimator=log_reg_model,
    X=X_train,
    y=y_train,
    cv=cv_strategy,
    scoring=scoring_metrics,
    n_jobs=-1,
    return_train_score=False
)

# Convert fold-level results into a clean table
log_reg_cv_table = pd.DataFrame({
    metric: log_reg_cv_results[f"test_{metric}"]
    for metric in scoring_metrics.keys()
})

log_reg_cv_table.index = [f"Fold {i}" for i in range(1, len(log_reg_cv_table))]

display(log_reg_cv_table.round(4))

# Create summary table
log_reg_cv_summary = pd.DataFrame({
    "metric": log_reg_cv_table.columns,
```

```

    "mean_score": log_reg_cv_table.mean().values,
    "std_score": log_reg_cv_table.std().values,
    "min_score": log_reg_cv_table.min().values,
    "max_score": log_reg_cv_table.max().values
})

display(log_reg_cv_summary.round(4))

print("Logistic Regression cross-validation completed successfully.")
print("The mean score shows average validation performance across folds.")
print("The standard deviation shows performance stability across folds.")

```

	accuracy	precision	recall	f1_score	roc_auc	pr_auc
Fold 1	0.8173	0.5579	0.7836	0.6518	0.8754	0.7245
Fold 2	0.8089	0.5455	0.7432	0.6292	0.8617	0.7067
Fold 3	0.8135	0.5513	0.7801	0.6460	0.8760	0.7243
Fold 4	0.8164	0.5560	0.7900	0.6526	0.8775	0.7333
Fold 5	0.8164	0.5548	0.8012	0.6556	0.8802	0.7327

	metric	mean_score	std_score	min_score	max_score
0	accuracy	0.8145	0.0034	0.8089	0.8173
1	precision	0.5531	0.0049	0.5455	0.5579
2	recall	0.7796	0.0219	0.7432	0.8012
3	f1_score	0.6471	0.0106	0.6292	0.6556
4	roc_auc	0.8742	0.0072	0.8617	0.8802
5	pr_auc	0.7243	0.0107	0.7067	0.7333

Logistic Regression cross-validation completed successfully.
The mean score shows average validation performance across folds.
The standard deviation shows performance stability across folds.

6.7 Threshold Analysis: Logistic Regression

By default, binary classification models use a probability threshold of 0.50.

However, in credit-risk modeling, the default threshold is not always the best business decision. A lower threshold may identify more potential defaulters, increasing recall, but it may also create more false positives by flagging more safe borrowers as risky.

This section compares several decision thresholds for the Logistic Regression baseline model.

The purpose is to understand the trade-off between:

- **False negatives:** risky borrowers incorrectly predicted as safe
- **False positives:** safe borrowers incorrectly predicted as risky
- **Recall:** ability to detect actual defaulters
- **Precision:** reliability of default predictions
- **F1-score:** balance between precision and recall

For this project, reducing false negatives is especially important because missed defaulters can create financial losses. However, precision also matters because rejecting too many safe borrowers can reduce business growth.

```
In [17]: # =====  
# Threshold Analysis: Logistic Regression  
# =====  
  
# Predicted probabilities for the default class  
log_reg_test_proba = log_reg_model.predict_proba(X_test)[:, 1]  
  
# Candidate thresholds to compare  
thresholds = [0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50]  
  
threshold_results = []  
  
for threshold in thresholds:  
    y_pred_threshold = (log_reg_test_proba >= threshold).astype(int)  
  
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred_threshold).ravel()  
  
    threshold_results.append({  
        "threshold": threshold,  
        "accuracy": accuracy_score(y_test, y_pred_threshold),  
        "precision": precision_score(y_test, y_pred_threshold, zero_division=0),  
        "recall": recall_score(y_test, y_pred_threshold, zero_division=0),  
        "f1_score": f1_score(y_test, y_pred_threshold, zero_division=0),  
        "true_negatives": tn,  
        "false_positives": fp,  
        "false_negatives": fn,  
        "true_positives": tp  
    })  
  
threshold_results_df = pd.DataFrame(threshold_results)  
  
display(threshold_results_df.round(4))  
  
# Plot threshold trade-offs  
plt.figure(figsize=(8, 5))  
plt.plot(threshold_results_df["threshold"], threshold_results_df["precision"],  
plt.plot(threshold_results_df["threshold"], threshold_results_df["recall"], ma  
plt.plot(threshold_results_df["threshold"], threshold_results_df["f1_score"],
```

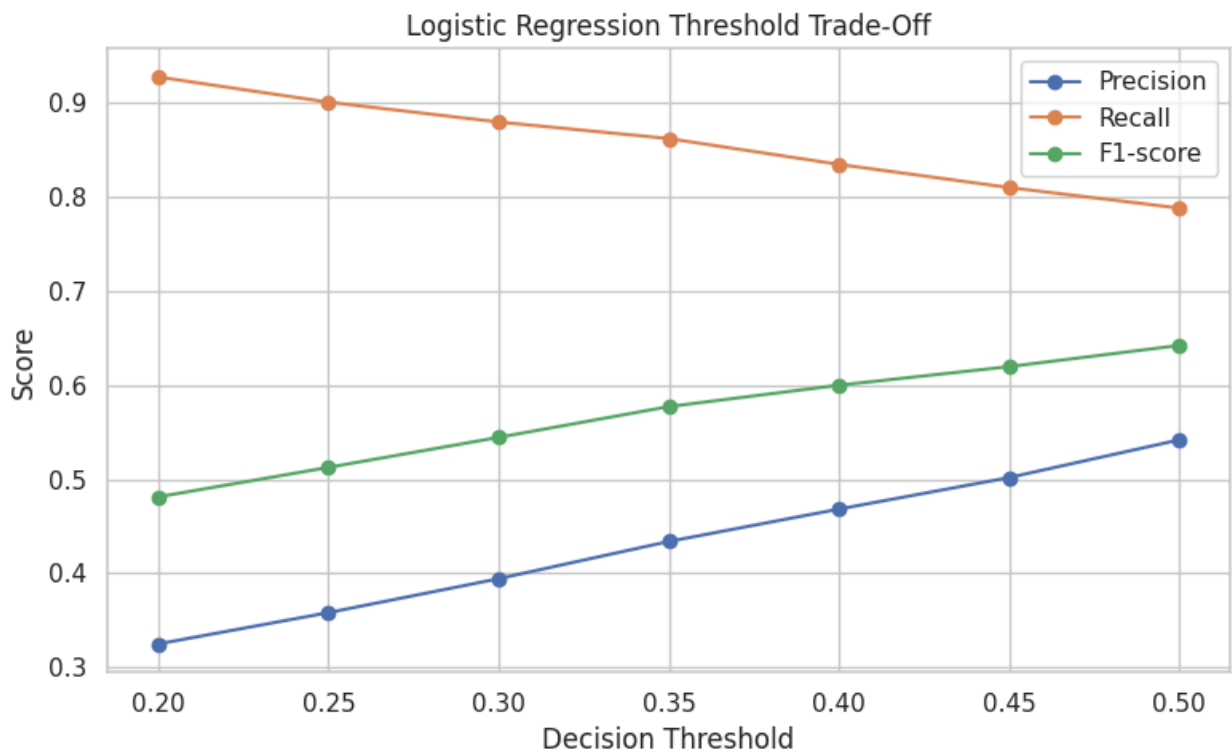
```
plt.title("Logistic Regression Threshold Trade-Off")
plt.xlabel("Decision Threshold")
plt.ylabel("Score")
plt.legend()
plt.tight_layout()
plt.show()

# Identify threshold with highest F1-score
best_f1_row = threshold_results_df.loc[threshold_results_df["f1_score"].idxmax()]

print("Best threshold by F1-score:")
display(pd.DataFrame([best_f1_row]).round(4))

print("Business interpretation:")
print("- Lower thresholds usually increase recall and reduce false negatives.")
print("- Higher thresholds usually increase precision and reduce false positives.")
print("- The final threshold should be selected based on business cost, not on accuracy alone.")
```

	threshold	accuracy	precision	recall	f1_score	true_negatives	false_positives
0	0.20	0.5636	0.3250	0.9276	0.4813	2353	2740
1	0.25	0.6261	0.3582	0.9008	0.5126	2798	2295
2	0.30	0.6787	0.3943	0.8797	0.5445	3171	1929
3	0.35	0.7243	0.4338	0.8622	0.5772	3493	1600
4	0.40	0.7570	0.4682	0.8347	0.5999	3745	1348
5	0.45	0.7830	0.5017	0.8101	0.6197	3949	1145
6	0.50	0.8083	0.5418	0.7883	0.6422	4145	945



Best threshold by F1-score:

	threshold	accuracy	precision	recall	f1_score	true_negatives	false_positives
6	0.5	0.8083	0.5418	0.7883	0.6422	4145.0	948.0

Business interpretation:

- Lower thresholds usually increase recall and reduce false negatives.
- Higher thresholds usually increase precision and reduce false positives.
- The final threshold should be selected based on business cost, not only statistical performance.

6.8 Business Interpretation: Logistic Regression Baseline

The Logistic Regression baseline provides a useful reference model for credit-risk classification.

The model achieved stable cross-validation performance, with an average ROC-AUC of approximately 0.87 and PR-AUC of approximately 0.72. This indicates that the model has a good ability to rank borrowers by default risk, even though the dataset is imbalanced.

Because `class_weight="balanced"` was used, the model gives more attention to the minority default class. This improves recall for default detection, but it also reduces precision because more borrowers are flagged as risky.

Threshold Trade-Off

The threshold analysis shows that changing the classification threshold has a major effect on business outcomes.

At a lower threshold, the model detects more actual defaulters:

- Threshold 0.20: recall = 0.9276
- Threshold 0.30: recall = 0.8797
- Threshold 0.40: recall = 0.8347
- Threshold 0.50: recall = 0.7883

However, lower thresholds also create more false positives. This means more borrowers who would not default are incorrectly flagged as risky.

At the default threshold of 0.50, the model produces the highest F1-score among the tested thresholds. However, it also misses 301 actual defaulters.

Business Meaning of False Negatives and False Positives

In this project:

- A **false negative** means the model predicts that a borrower is safe, but the borrower actually defaults.
- A **false positive** means the model predicts that a borrower may default, but the borrower would actually repay.

False negatives are usually more costly in credit-risk settings because they may lead to approving loans that later default.

False positives are also important because they may lead to rejecting or manually reviewing good customers, which can reduce revenue and customer growth.

Recommended Interpretation

The Logistic Regression model should be treated as an interpretable baseline rather than the final model.

A threshold around 0.40 may be a reasonable business-review threshold because it reduces false negatives compared with 0.50 while avoiding the very large number of false positives seen at lower thresholds such as 0.20 or 0.30.

However, the final threshold should be selected after comparing Logistic

Regression with Random Forest and after running a business cost simulation.

The next modeling step is to train and evaluate a Random Forest model to determine whether a non-linear model can improve predictive performance and reduce business risk.

6.9 Advanced Model: Random Forest

Random Forest is used as the advanced model because it can capture non-linear relationships and interactions between borrower, loan, and credit-history variables.

Compared with Logistic Regression, Random Forest can better model complex risk patterns such as:

- Interactions between income and loan amount
- Risk differences across loan grades
- Non-linear relationships between interest rate and default probability
- Combined effects of borrower profile and credit history

The model is built inside a scikit-learn pipeline using the tree-based preprocessing pipeline:

- Numerical features are passed through without scaling.
- Categorical features are one-hot encoded.
- A Random Forest classifier is trained on the processed features.

Because the dataset is imbalanced, `class_weight="balanced_subsample"` is used. This helps the model give more attention to the minority default class while building individual trees.

The model is first evaluated using the default threshold of 0.50. Later, it will be compared with the tuned Random Forest model and evaluated using threshold analysis and business cost simulation.

```
In [18]: # =====  
# Advanced Model: Random Forest Baseline  
# =====  
  
# Create Random Forest pipeline  
rf_model = Pipeline(steps=[  
    ("preprocessor", preprocessor_tree),  
    ("model", RandomForestClassifier(  
        n_estimators=200,  
        max_depth=None,  
        min_samples_leaf=2,  
        class_weight="balanced_subsample",
```

```

        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
])

# Train model
rf_model.fit(X_train, y_train)

print("Random Forest baseline model trained successfully.")

# Evaluate model on test set using default threshold
rf_metrics = evaluate_classification_model(
    model=rf_model,
    X_data=X_test,
    y_true=y_test,
    model_name="Random Forest Baseline",
    threshold=0.50
)

# Store model result without duplicating if the cell is rerun
if "model_results" not in globals():
    model_results = []

model_results = [
    result for result in model_results
    if result["model"] != "Random Forest Baseline"
]

model_results.append(rf_metrics)

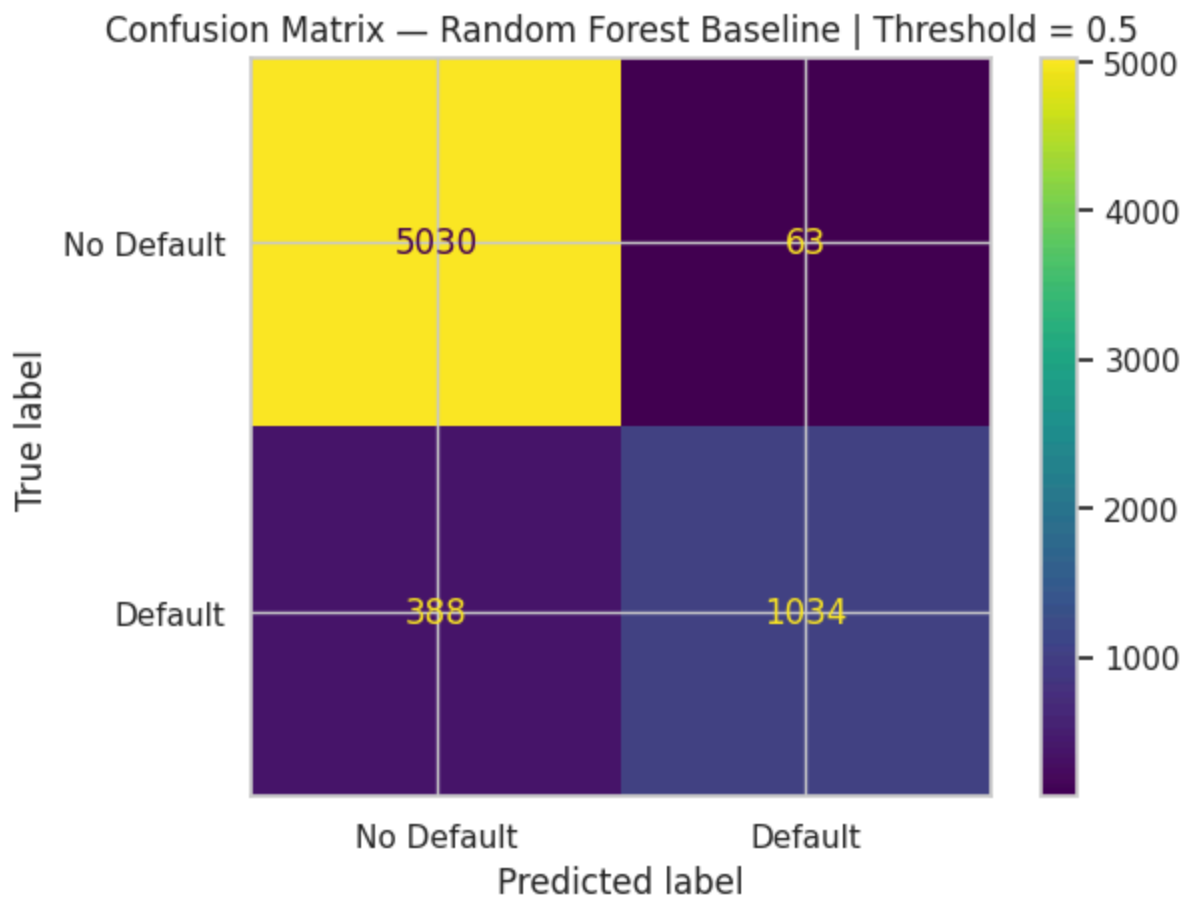
```

Random Forest baseline model trained successfully.

	model	threshold	accuracy	precision	recall	f1_score	roc_auc	pr_auc
0	Random Forest Baseline	0.5	0.930775	0.942571	0.727145	0.820961	0.930451	0.88103

Classification Report: Random Forest Baseline

	precision	recall	f1-score	support
No Default	0.93	0.99	0.96	5093
Default	0.94	0.73	0.82	1422
accuracy			0.93	6515
macro avg	0.94	0.86	0.89	6515
weighted avg	0.93	0.93	0.93	6515



6.10 Cross-Validation: Random Forest Baseline

The Random Forest baseline was first evaluated on the holdout test set. To assess model stability more robustly, stratified cross-validation is applied on the training data.

Cross-validation helps determine whether the Random Forest model performs consistently across different folds or whether the result is strongly dependent on one specific train-test split.

The same 5-fold stratified cross-validation strategy is used so that the class distribution is preserved in every fold.

The model is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

- PR-AUC

This step helps compare Random Forest against Logistic Regression in a more reliable way before hyperparameter tuning.

```
In [19]: # =====
# Cross-Validation: Random Forest Baseline
# =====

# Run cross-validation on the training data
# n_jobs is kept at 1 here because the Random Forest model itself already uses
rf_cv_results = cross_validate(
    estimator=rf_model,
    X=X_train,
    y=y_train,
    cv=cv_strategy,
    scoring=scoring_metrics,
    n_jobs=1,
    return_train_score=False
)

# Convert fold-level results into a clean table
rf_cv_table = pd.DataFrame({
    metric: rf_cv_results[f"test_{metric}"]
    for metric in scoring_metrics.keys()
})

rf_cv_table.index = [f"Fold {i}" for i in range(1, len(rf_cv_table) + 1)]

display(rf_cv_table.round(4))

# Create summary table
rf_cv_summary = pd.DataFrame({
    "metric": rf_cv_table.columns,
    "mean_score": rf_cv_table.mean().values,
    "std_score": rf_cv_table.std().values,
    "min_score": rf_cv_table.min().values,
    "max_score": rf_cv_table.max().values
})

display(rf_cv_summary.round(4))

print("Random Forest baseline cross-validation completed successfully.")
print("The mean score shows average validation performance across folds.")
print("The standard deviation shows performance stability across folds.")
```

	accuracy	precision	recall	f1_score	roc_auc	pr_auc
Fold 1	0.9309	0.9312	0.7379	0.8234	0.9356	0.8904
Fold 2	0.9244	0.9335	0.7036	0.8024	0.9244	0.8726
Fold 3	0.9348	0.9443	0.7449	0.8328	0.9363	0.8928
Fold 4	0.9305	0.9283	0.7390	0.8229	0.9331	0.8869
Fold 5	0.9301	0.9479	0.7194	0.8180	0.9276	0.8812

	metric	mean_score	std_score	min_score	max_score
0	accuracy	0.9302	0.0037	0.9244	0.9348
1	precision	0.9370	0.0086	0.9283	0.9479
2	recall	0.7290	0.0171	0.7036	0.7449
3	f1_score	0.8199	0.0112	0.8024	0.8328
4	roc_auc	0.9314	0.0052	0.9244	0.9363
5	pr_auc	0.8848	0.0081	0.8726	0.8928

Random Forest baseline cross-validation completed successfully.
The mean score shows average validation performance across folds.
The standard deviation shows performance stability across folds.

6.11 Hyperparameter Tuning: Random Forest

The Random Forest baseline model already performs strongly, but hyperparameter tuning is used to check whether performance can be improved further.

Hyperparameter tuning helps optimize model settings such as:

- Number of trees
- Maximum tree depth
- Minimum samples required in leaf nodes
- Number of features considered at each split

Because this is a credit-risk classification problem, the tuning process should not focus only on accuracy. The model should balance default detection with precision.

In this step, `GridSearchCV` is used with stratified 5-fold cross-validation. The tuning process evaluates multiple metrics, but the best model is selected using F1-score because F1-score balances precision and recall.

The tuned Random Forest model will later be evaluated on the holdout test set and compared with the baseline Random Forest.

```

In [20]: # =====
# Hyperparameter Tuning: Random Forest
# =====

# Define Random Forest pipeline for tuning
rf_tuning_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor_tree),
    ("model", RandomForestClassifier(
        class_weight="balanced_subsample",
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
])

# Define a compact but meaningful parameter grid
rf_param_grid = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [10, 15, None],
    "model__min_samples_leaf": [1, 2, 5],
    "model__max_features": ["sqrt"]
}

# Grid search with multiple scoring metrics
rf_grid_search = GridSearchCV(
    estimator=rf_tuning_pipeline,
    param_grid=rf_param_grid,
    scoring=scoring_metrics,
    refit="f1_score",
    cv=cv_strategy,
    n_jobs=1,
    verbose=1,
    return_train_score=True
)

# Fit grid search on training data
rf_grid_search.fit(X_train, y_train)

print("Random Forest hyperparameter tuning completed successfully.")
print(f"Best cross-validation F1-score: {rf_grid_search.best_score_:.4f}")
print("Best parameters:")
print(rf_grid_search.best_params_)

# Convert tuning results into a readable table
rf_tuning_results = pd.DataFrame(rf_grid_search.cv_results_)

selected_tuning_columns = [
    "mean_test_accuracy",
    "mean_test_precision",
    "mean_test_recall",
    "mean_test_f1_score",
    "mean_test_roc_auc",
    "mean_test_pr_auc",
    "std_test_f1_score",

```

```

        "param_model__n_estimators",
        "param_model__max_depth",
        "param_model__min_samples_leaf",
        "param_model__max_features"
    ]

    rf_tuning_summary = (
        rf_tuning_results[selected_tuning_columns]
        .sort_values(by="mean_test_f1_score", ascending=False)
        .head(10)
    )

    display(rf_tuning_summary.round(4))

# Store best tuned model
    rf_tuned_model = rf_grid_search.best_estimator_

# Evaluate tuned model on test set
    rf_tuned_metrics = evaluate_classification_model(
        model=rf_tuned_model,
        X_data=X_test,
        y_true=y_test,
        model_name="Random Forest Tuned",
        threshold=0.50
    )

# Store tuned model result without duplicating if the cell is rerun
    if "model_results" not in globals():
        model_results = []

    model_results = [
        result for result in model_results
        if result["model"] != "Random Forest Tuned"
    ]

    model_results.append(rf_tuned_metrics)

```

Fitting 5 folds for each of 18 candidates, totalling 90 fits

Random Forest hyperparameter tuning completed successfully.

Best cross-validation F1-score: 0.8232

Best parameters:

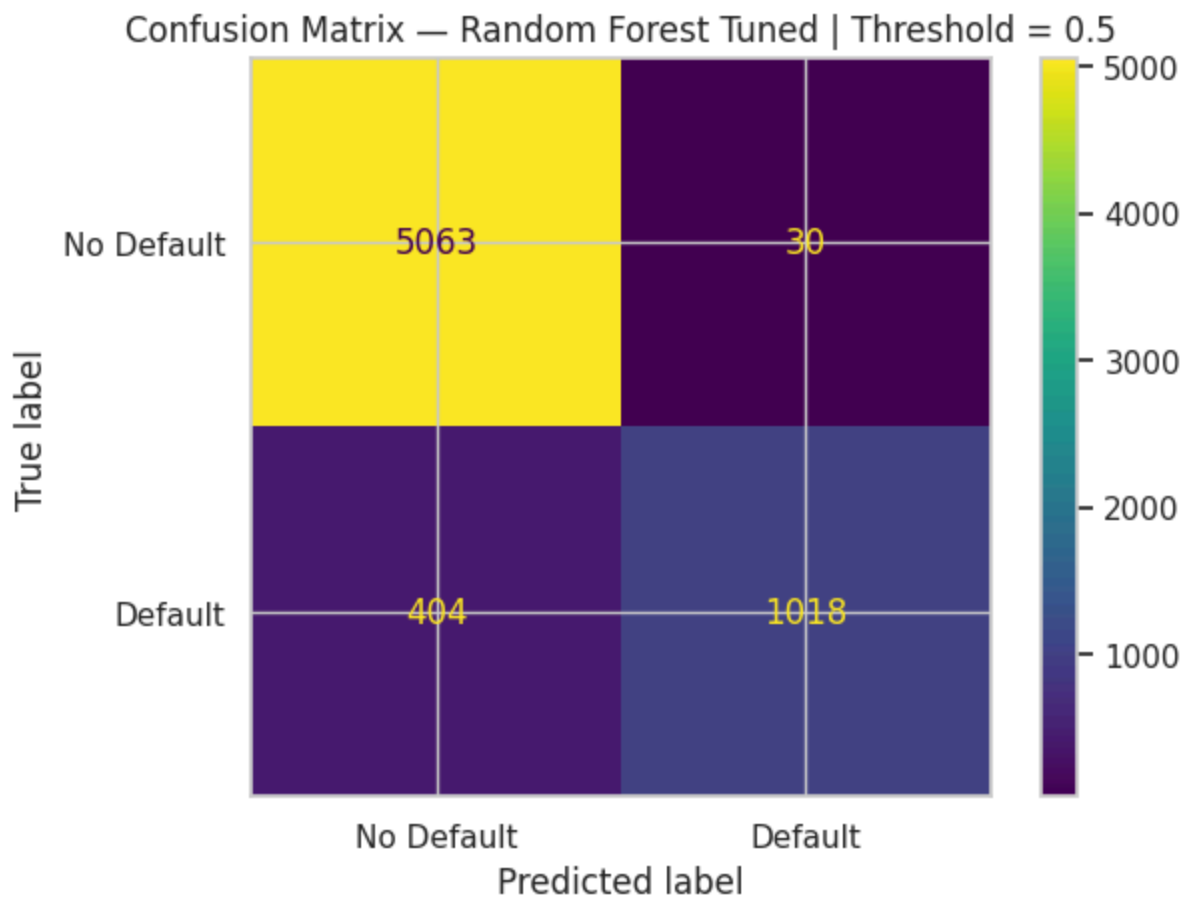
```
{'model__max_depth': None, 'model__max_features': 'sqrt', 'model__min_samples_
leaf': 1, 'model__n_estimators': 100}
```

	mean_test_accuracy	mean_test_precision	mean_test_recall	mean_test_f1_score
12	0.9331	0.9716	0.7144	0.823
13	0.9330	0.9702	0.7147	0.823
7	0.9316	0.9530	0.7223	0.823
6	0.9312	0.9508	0.7223	0.820
14	0.9303	0.9367	0.7299	0.820
15	0.9302	0.9370	0.7290	0.819
8	0.9278	0.9197	0.7332	0.819
9	0.9279	0.9217	0.7318	0.819
17	0.9211	0.8737	0.7466	0.809
11	0.9198	0.8661	0.7483	0.807

	model	threshold	accuracy	precision	recall	f1_score	roc_auc	pr_auc
0	Random Forest Tuned	0.5	0.933384	0.971374	0.715893	0.824291	0.926705	0.878141

Classification Report: Random Forest Tuned

	precision	recall	f1-score	support
No Default	0.93	0.99	0.96	5093
Default	0.97	0.72	0.82	1422
accuracy			0.93	6515
macro avg	0.95	0.86	0.89	6515
weighted avg	0.94	0.93	0.93	6515



6.12 Model Comparison: Baseline and Tuned Models

This section compares the main models evaluated in the project:

1. Logistic Regression Baseline
2. Random Forest Baseline
3. Random Forest Tuned

The comparison focuses on both statistical performance and business relevance.

The key business trade-off is between:

- **False negatives:** actual defaulters predicted as safe
- **False positives:** safe borrowers predicted as risky

In credit-risk modeling, false negatives are usually more costly because they may lead to approving loans that later default. However, false positives also matter because they may lead to unnecessary rejection or manual review of good customers.

Therefore, the final model should not be selected using accuracy alone. It should

be selected based on a combination of:

- Recall
- Precision
- F1-score
- ROC-AUC
- PR-AUC
- Confusion matrix values
- Business cost simulation

```
In [21]: # =====  
# Model Comparison Table  
# =====  
  
# Convert stored model results into a comparison table  
model_comparison_df = pd.DataFrame(model_results)  
  
# Select and order important columns  
comparison_columns = [  
    "model",  
    "threshold",  
    "accuracy",  
    "precision",  
    "recall",  
    "f1_score",  
    "roc_auc",  
    "pr_auc",  
    "false_positives",  
    "false_negatives",  
    "true_positives",  
    "true_negatives"  
]  
  
model_comparison_df = model_comparison_df[comparison_columns]  
  
# Sort by F1-score for statistical comparison  
model_comparison_sorted = model_comparison_df.sort_values(  
    by="f1_score",  
    ascending=False  
)  
  
display(model_comparison_sorted.round(4))  
  
# Visual comparison of key metrics  
metric_columns = ["accuracy", "precision", "recall", "f1_score", "roc_auc", "pr_auc"]  
  
model_comparison_plot = model_comparison_df.set_index("model")[metric_columns]  
  
ax = model_comparison_plot.plot(  
    kind="bar",  
    figsize=(10, 6))
```



```

)

ax.set_title("Model Performance Comparison")
ax.set_xlabel("Model")
ax.set_ylabel("Score")
ax.tick_params(axis="x", rotation=30)
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()

# Business-focused comparison of false positives and false negatives
error_comparison = model_comparison_df[[
    "model",
    "false_positives",
    "false_negatives"
]].copy()

display(error_comparison)

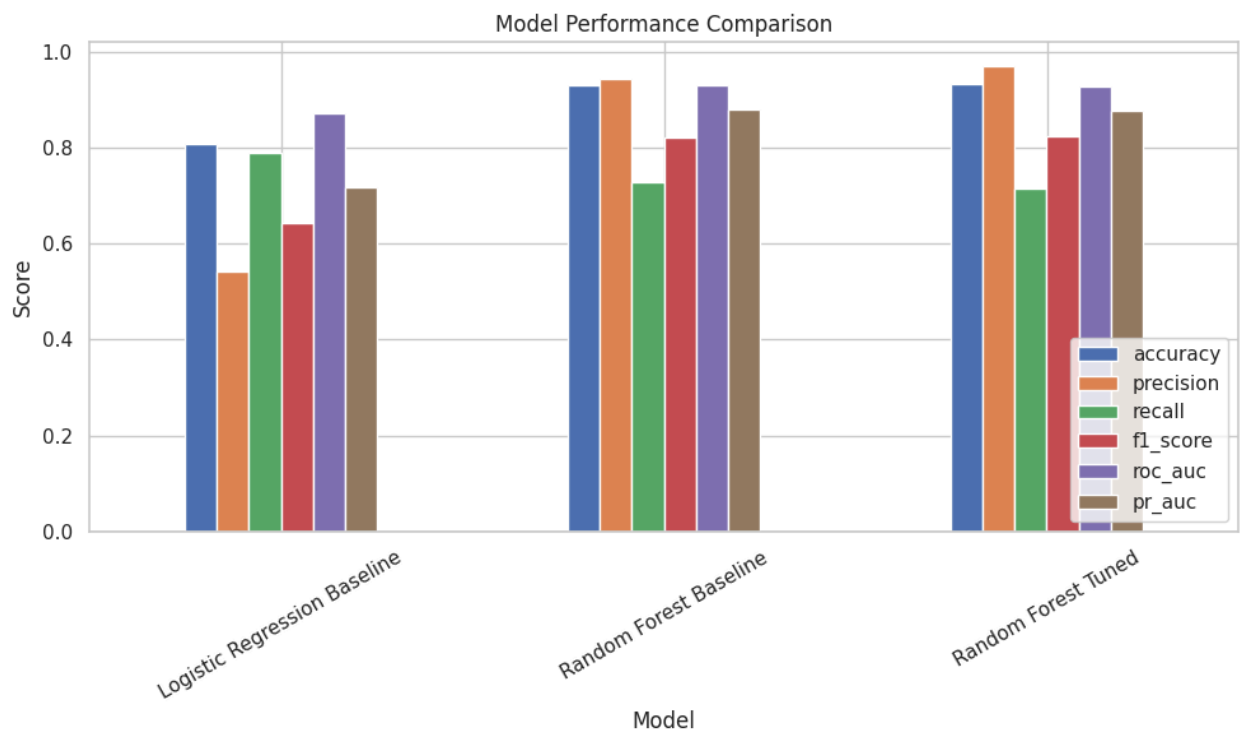
ax = error_comparison.set_index("model").plot(
    kind="bar",
    figsize=(8, 5)
)

ax.set_title("False Positives vs False Negatives by Model")
ax.set_xlabel("Model")
ax.set_ylabel("Number of Cases")
ax.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.show()

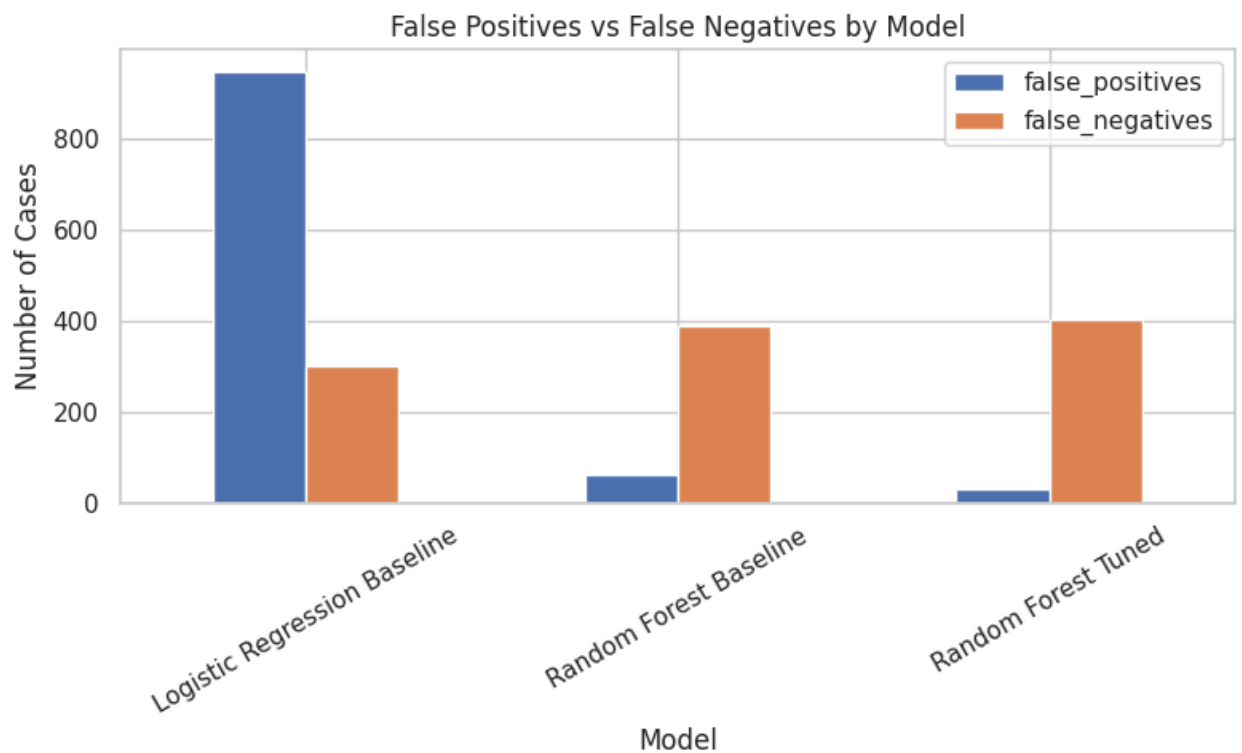
print("Model comparison completed successfully.")

```

	model	threshold	accuracy	precision	recall	f1_score	roc_auc	pr_auc	fa
2	Random Forest Tuned	0.5	0.9334	0.9714	0.7159	0.8243	0.9267	0.8781	
1	Random Forest Baseline	0.5	0.9308	0.9426	0.7271	0.8210	0.9305	0.8810	
0	Logistic Regression Baseline	0.5	0.8083	0.5418	0.7883	0.6422	0.8710	0.7165	



	model	false_positives	false_negatives
0	Logistic Regression Baseline	948	301
1	Random Forest Baseline	63	388
2	Random Forest Tuned	30	404



Model comparison completed successfully.

6.13 Business Interpretation of Model Comparison

The model comparison shows that each model has a different business risk profile.

1. Logistic Regression Baseline

The Logistic Regression baseline achieved the highest recall among the three models at the default threshold.

- Recall: 0.7883
- False negatives: 301
- False positives: 948

This means Logistic Regression detects more actual defaulters than the Random Forest models. However, it also incorrectly flags many non-default borrowers as risky.

From a business perspective, this may create too many unnecessary manual reviews or rejected applications. Although the model is useful as an interpretable baseline, its low precision makes it less attractive as the final decision-support model.

2. Random Forest Baseline

The Random Forest baseline achieved a strong balance between predictive performance and business usability.

- Precision: 0.9426
- Recall: 0.7271
- F1-score: 0.8210
- ROC-AUC: 0.9305
- PR-AUC: 0.8810
- False positives: 63
- False negatives: 388

Compared with Logistic Regression, the Random Forest baseline greatly reduces false positives while maintaining strong overall model performance.

This means that when the Random Forest baseline predicts default, the prediction is usually reliable. However, it misses more actual defaulters than Logistic Regression at the default threshold.

3. Tuned Random Forest

The tuned Random Forest achieved the highest precision and slightly highest F1-score.

- Precision: 0.9714
- Recall: 0.7159
- F1-score: 0.8243
- False positives: 30
- False negatives: 404

This model is more conservative. It produces very few false positives, meaning it rarely flags safe borrowers as risky. However, it also misses more actual defaulters than the Random Forest baseline.

From a credit-risk perspective, this is not automatically better. A model with extremely high precision but lower recall may be too conservative in flagging risk.

4. Candidate Model Before Cost Simulation

Based on the current results, the Random Forest baseline is a strong candidate model because it provides:

- High precision
- Strong ROC-AUC and PR-AUC
- Good F1-score
- Fewer false negatives than the tuned Random Forest
- Much fewer false positives than Logistic Regression

However, the final model should not be selected only from statistical metrics.

The next step is to perform threshold analysis and business cost simulation. This will help determine whether the Random Forest baseline or tuned Random Forest provides better business value under realistic assumptions about the cost of false positives and false negatives.

For credit-risk decision support, the best model is not necessarily the model with the highest accuracy or F1-score. The best model is the one that provides the most useful balance between risk reduction and business opportunity.

6.14 Random Forest Threshold Analysis

The previous model comparison evaluated Random Forest models using the default

threshold of 0.50.

However, credit-risk decisions often require a threshold that reflects business risk. A lower threshold may detect more actual defaulters, reducing false negatives, but it may also increase false positives by flagging more safe borrowers as risky.

This section compares several decision thresholds for both Random Forest models:

1. Random Forest Baseline
2. Random Forest Tuned

The goal is to evaluate whether threshold adjustment can improve the business usefulness of the Random Forest models.

The analysis focuses on:

- Precision
- Recall
- F1-score
- False positives
- False negatives
- True positives
- True negatives

This prepares the project for the next step: business cost simulation.

```
In [22]: # =====
# Random Forest Threshold Analysis
# =====

def threshold_analysis(model, X_data, y_true, model_name, thresholds):
    """
    Evaluate a fitted model across multiple classification thresholds.
    """
    y_proba = model.predict_proba(X_data)[:, 1]
    results = []

    for threshold in thresholds:
        y_pred = (y_proba >= threshold).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

        results.append({
            "model": model_name,
            "threshold": threshold,
            "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, zero_division=0),
            "recall": recall_score(y_true, y_pred, zero_division=0),
            "f1_score": f1_score(y_true, y_pred, zero_division=0),
            "false_positives": fp,
```

```

        "false_negatives": fn,
        "true_positives": tp,
        "true_negatives": tn
    })

    return pd.DataFrame(results)

# Candidate thresholds
rf_thresholds = [0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50]

# Run threshold analysis for both Random Forest models
rf_baseline_threshold_df = threshold_analysis(
    model=rf_model,
    X_data=X_test,
    y_true=y_test,
    model_name="Random Forest Baseline",
    thresholds=rf_thresholds
)

rf_tuned_threshold_df = threshold_analysis(
    model=rf_tuned_model,
    X_data=X_test,
    y_true=y_test,
    model_name="Random Forest Tuned",
    thresholds=rf_thresholds
)

# Combine results
rf_threshold_comparison_df = pd.concat(
    [rf_baseline_threshold_df, rf_tuned_threshold_df],
    ignore_index=True
)

display(rf_threshold_comparison_df.round(4))

# Best threshold by F1-score for each model
best_rf_thresholds_by_f1 = (
    rf_threshold_comparison_df
    .loc[rf_threshold_comparison_df.groupby("model")["f1_score"].idxmax()]
    .reset_index(drop=True)
)

print("Best Random Forest thresholds by F1-score:")
display(best_rf_thresholds_by_f1.round(4))

# Business-focused subset
business_threshold_view = rf_threshold_comparison_df[[
    "model",
    "threshold",
    "precision",
    "recall",
    "f1_score",

```

```

        "false_positives",
        "false_negatives"
    ]].copy()

display(business_threshold_view.round(4))

# Plot threshold trade-offs for each model
for model_name in rf_threshold_comparison_df["model"].unique():
    plot_df = rf_threshold_comparison_df[
        rf_threshold_comparison_df["model"] == model_name
    ]

    ax = plot_df.plot(
        x="threshold",
        y=["precision", "recall", "f1_score"],
        marker="o",
        figsize=(8, 5)
    )

    ax.set_title(f"Threshold Trade-Off – {model_name}")
    ax.set_xlabel("Decision Threshold")
    ax.set_ylabel("Score")
    ax.set_ylim(0, 1.05)
    plt.tight_layout()
    plt.show()

# Plot false positives and false negatives by threshold
for model_name in rf_threshold_comparison_df["model"].unique():
    plot_df = rf_threshold_comparison_df[
        rf_threshold_comparison_df["model"] == model_name
    ]

    ax = plot_df.plot(
        x="threshold",
        y=["false_positives", "false_negatives"],
        marker="o",
        figsize=(8, 5)
    )

    ax.set_title(f"False Positives vs False Negatives – {model_name}")
    ax.set_xlabel("Decision Threshold")
    ax.set_ylabel("Number of Cases")
    plt.tight_layout()
    plt.show()

print("Random Forest threshold analysis completed successfully.")

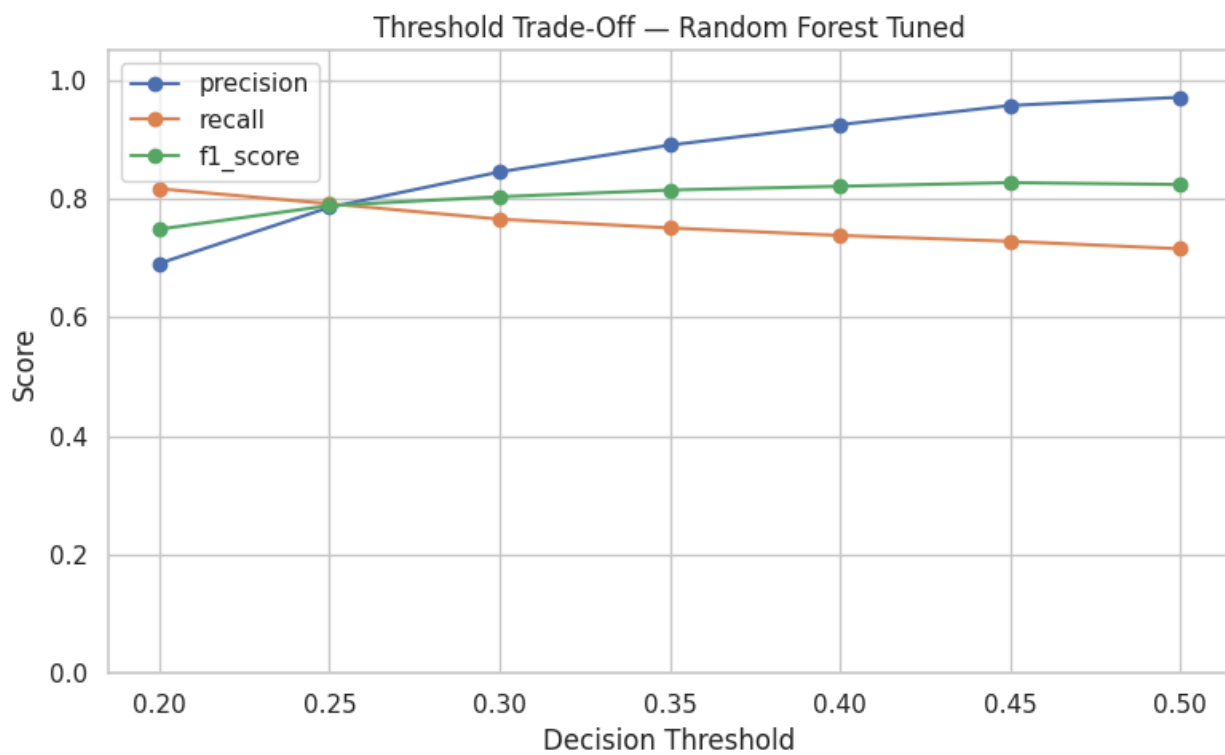
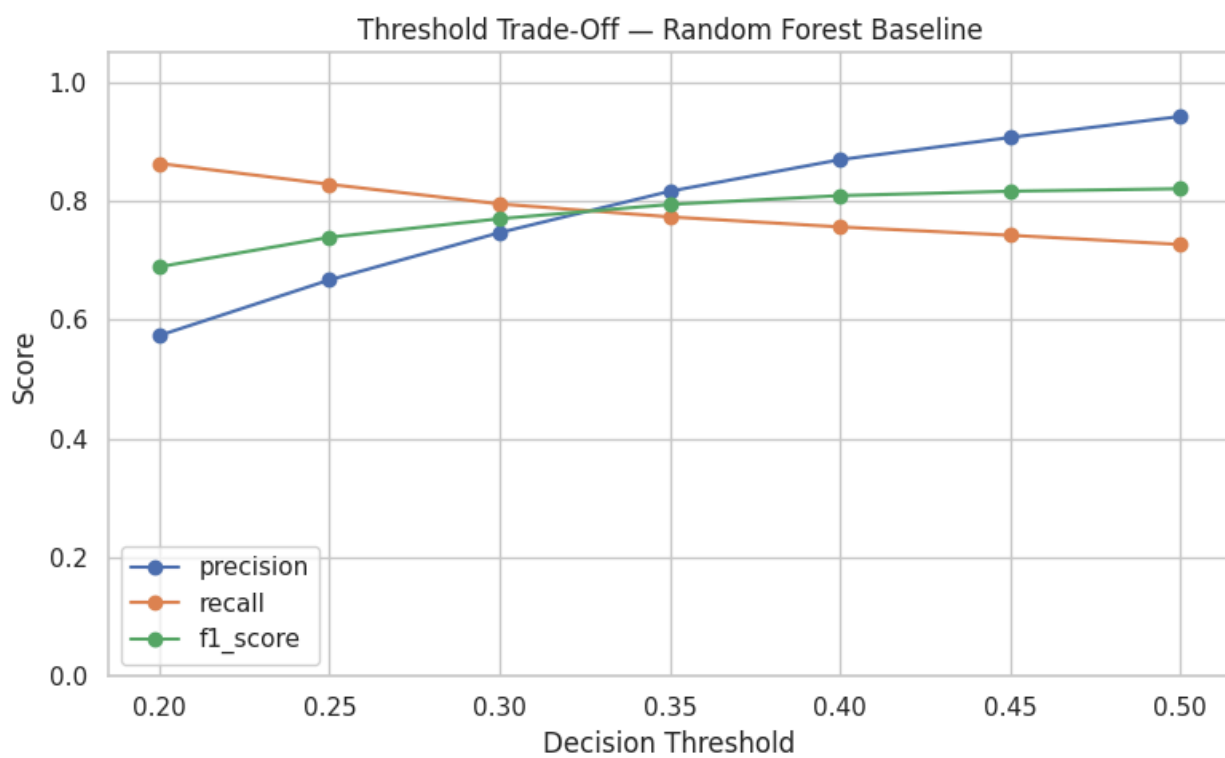
```

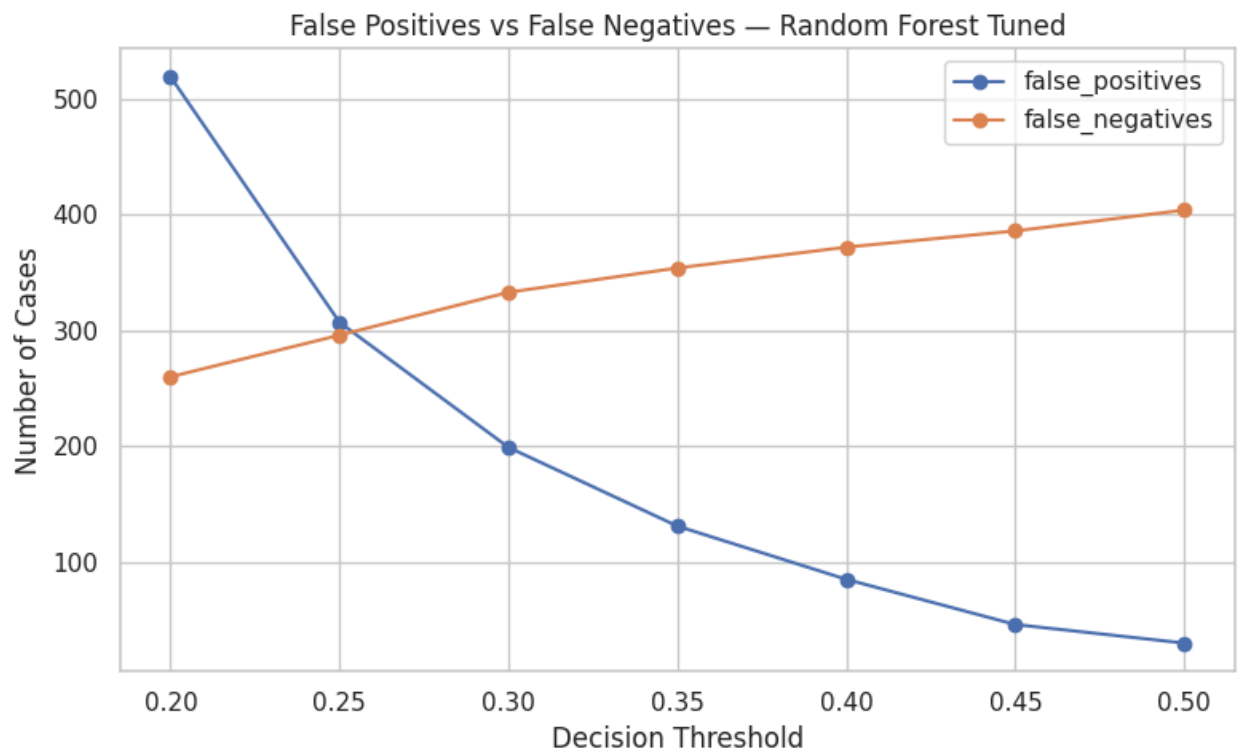
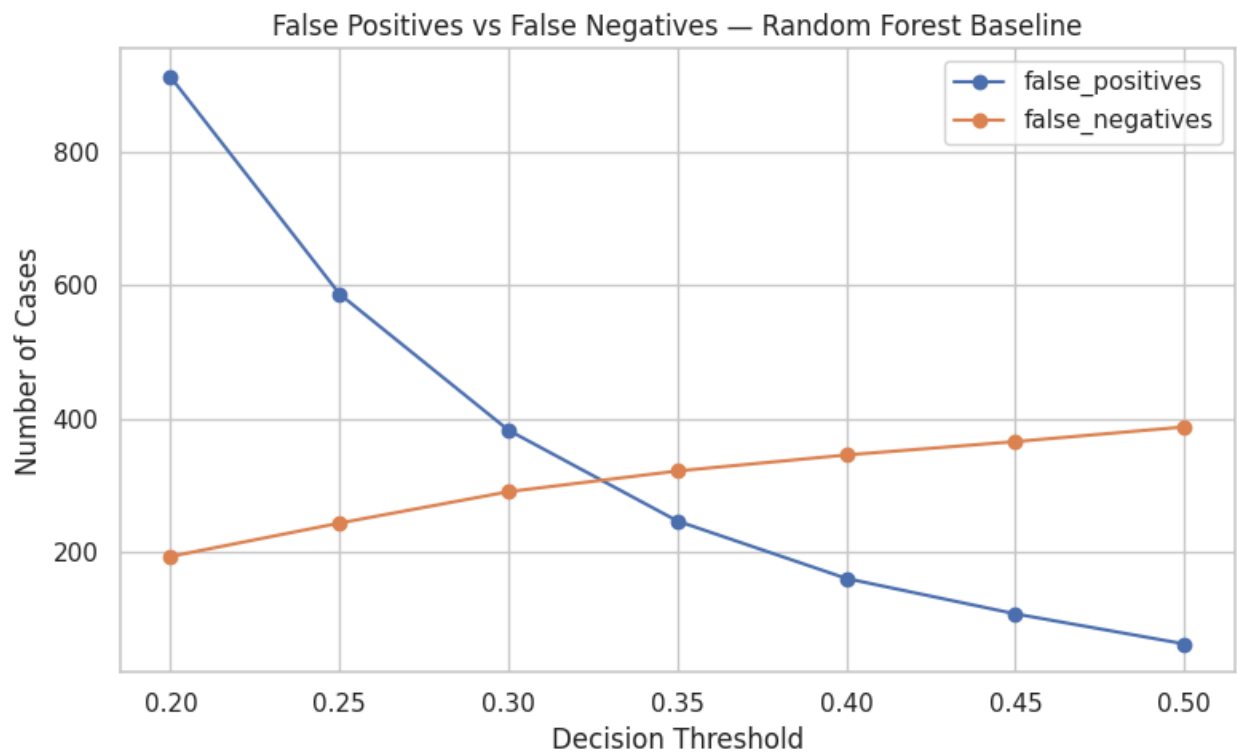
	model	threshold	accuracy	precision	recall	f1_score	false_positives	fals
0	Random Forest Baseline	0.20	0.8302	0.5738	0.8636	0.6895	912	
1	Random Forest Baseline	0.25	0.8724	0.6674	0.8284	0.7393	587	
2	Random Forest Baseline	0.30	0.8965	0.7470	0.7954	0.7704	383	
3	Random Forest Baseline	0.35	0.9127	0.8166	0.7736	0.7945	247	
4	Random Forest Baseline	0.40	0.9222	0.8698	0.7567	0.8093	161	
5	Random Forest Baseline	0.45	0.9272	0.9072	0.7426	0.8167	108	
6	Random Forest Baseline	0.50	0.9308	0.9426	0.7271	0.8210	63	
7	Random Forest Tuned	0.20	0.8804	0.6913	0.8172	0.7490	519	
8	Random Forest Tuned	0.25	0.9074	0.7858	0.7918	0.7888	307	
9	Random Forest Tuned	0.30	0.9183	0.8455	0.7658	0.8037	199	
10	Random Forest Tuned	0.35	0.9256	0.8907	0.7511	0.8150	131	
11	Random Forest Tuned	0.40	0.9299	0.9251	0.7384	0.8213	85	
12	Random Forest Tuned	0.45	0.9337	0.9575	0.7286	0.8275	46	
13	Random Forest Tuned	0.50	0.9334	0.9714	0.7159	0.8243	30	

Best Random Forest thresholds by F1-score:

	model	threshold	accuracy	precision	recall	f1_score	false_positives	false
0	Random Forest Baseline	0.50	0.9308	0.9426	0.7271	0.8210	63	
1	Random Forest Tuned	0.45	0.9337	0.9575	0.7286	0.8275	46	

	model	threshold	precision	recall	f1_score	false_positives	false_negative
0	Random Forest Baseline	0.20	0.5738	0.8636	0.6895	912	19
1	Random Forest Baseline	0.25	0.6674	0.8284	0.7393	587	24
2	Random Forest Baseline	0.30	0.7470	0.7954	0.7704	383	29
3	Random Forest Baseline	0.35	0.8166	0.7736	0.7945	247	32
4	Random Forest Baseline	0.40	0.8698	0.7567	0.8093	161	34
5	Random Forest Baseline	0.45	0.9072	0.7426	0.8167	108	36
6	Random Forest Baseline	0.50	0.9426	0.7271	0.8210	63	38
7	Random Forest Tuned	0.20	0.6913	0.8172	0.7490	519	26
8	Random Forest Tuned	0.25	0.7858	0.7918	0.7888	307	29
9	Random Forest Tuned	0.30	0.8455	0.7658	0.8037	199	33
10	Random Forest Tuned	0.35	0.8907	0.7511	0.8150	131	35
11	Random Forest Tuned	0.40	0.9251	0.7384	0.8213	85	37
12	Random Forest Tuned	0.45	0.9575	0.7286	0.8275	46	38
13	Random Forest Tuned	0.50	0.9714	0.7159	0.8243	30	40





Random Forest threshold analysis completed successfully.

6.15 Business Interpretation of Random Forest Threshold Analysis

The Random Forest threshold analysis shows how different decision thresholds

affect business risk.

At lower thresholds, the models classify more borrowers as potential defaulters. This increases recall and reduces false negatives, but it also increases false positives.

At higher thresholds, the models become more conservative. This improves precision and reduces false positives, but it increases false negatives by missing more actual defaulters.

1. Random Forest Baseline

For the Random Forest baseline, the best threshold by F1-score is 0.50.

At threshold 0.50, the model achieves:

- Precision: 0.9426
- Recall: 0.7271
- F1-score: 0.8210
- False positives: 63
- False negatives: 388

This means the Random Forest baseline is highly reliable when it predicts default. However, it still misses 388 actual defaulters.

Lowering the threshold improves recall. For example, at threshold 0.30:

- Recall increases to 0.7954
- False negatives decrease to 291
- False positives increase to 383

This may be useful if the business wants to reduce missed defaulters and is willing to review more applications manually.

2. Tuned Random Forest

For the tuned Random Forest, the best threshold by F1-score is 0.45.

At threshold 0.45, the model achieves:

- Precision: 0.9575
- Recall: 0.7286
- F1-score: 0.8275
- False positives: 46

- False negatives: 386

This is slightly better than the tuned model at threshold 0.50 because it reduces false negatives while maintaining very high precision.

The tuned Random Forest at threshold 0.45 appears to be the strongest statistical candidate so far because it has the highest F1-score and keeps false positives low.

3. Business Trade-Off

The threshold analysis confirms that model performance depends strongly on the selected threshold.

There is no universally correct threshold. The best threshold depends on the business cost of:

- Approving a borrower who later defaults
- Rejecting or reviewing a borrower who would have repaid

For example:

- A conservative lender may prefer a lower threshold to reduce missed defaulters.
 - A growth-focused lender may prefer a higher threshold to avoid rejecting too many good borrowers.
 - A balanced decision-support system may use probability bands instead of a single automatic approve/reject rule.
-

4. Candidate for Cost Simulation

Based on the threshold analysis, the most relevant candidates for business cost simulation are:

1. Logistic Regression Baseline at threshold 0.50
2. Random Forest Baseline at threshold 0.50
3. Random Forest Baseline at threshold 0.30
4. Tuned Random Forest at threshold 0.45
5. Tuned Random Forest at threshold 0.50

These candidates represent different trade-offs between default detection and unnecessary risk flags.

The next step is to run a business cost simulation to estimate which model-

threshold combination provides the best business value under assumed costs for false positives and false negatives.

6.16 Business Cost Simulation

Statistical metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC are useful for evaluating model performance. However, credit-risk decisions also require business-cost reasoning.

In this project, the two main prediction errors have different business consequences:

1. False Negative

- The model predicts that a borrower is safe.
- The borrower actually defaults.
- This may lead to financial loss because a risky loan is approved.

2. False Positive

- The model predicts that a borrower is risky.
- The borrower would actually repay.
- This may create lost revenue opportunity or unnecessary manual review.

Because false negatives are usually more costly in credit-risk settings, this simulation assumes the following relative cost structure:

Error Type	Assumed Cost
False Negative	10 cost units
False Positive	1 cost unit

These are simplified hypothetical values. In a real financial institution, the actual costs would depend on loan amount, interest income, recovery rate, operational review cost, and risk policy.

The goal of this simulation is not to estimate exact financial profit. The goal is to compare model-threshold combinations under a business-oriented cost assumption.

```
In [23]: # =====  
# Business Cost Simulation  
# =====
```

```

# Define hypothetical relative costs
COST_FALSE_NEGATIVE = 10
COST_FALSE_POSITIVE = 1

print("Cost assumptions:")
print(f"False Negative cost: {COST_FALSE_NEGATIVE} cost units")
print(f"False Positive cost: {COST_FALSE_POSITIVE} cost unit")

# -----
# Helper function for cost calculation
# -----

def calculate_business_cost(false_positives, false_negatives,
                             cost_fp=COST_FALSE_POSITIVE,
                             cost_fn=COST_FALSE_NEGATIVE):
    """
    Calculate total business cost from false positives and false negatives.
    """
    return (false_positives * cost_fp) + (false_negatives * cost_fn)

# -----
# Candidate 1: Logistic Regression threshold results
# -----

log_reg_cost_candidates = threshold_results_df.copy()
log_reg_cost_candidates["model"] = "Logistic Regression Baseline"

log_reg_cost_candidates = log_reg_cost_candidates[
    log_reg_cost_candidates["threshold"].isin([0.40, 0.50])
].copy()

# -----
# Candidate 2: Random Forest threshold results
# -----

rf_cost_candidates = rf_threshold_comparison_df[
    (
        (rf_threshold_comparison_df["model"] == "Random Forest Baseline") &
        (rf_threshold_comparison_df["threshold"].isin([0.30, 0.40, 0.50]))
    )
    |
    (
        (rf_threshold_comparison_df["model"] == "Random Forest Tuned") &
        (rf_threshold_comparison_df["threshold"].isin([0.35, 0.45, 0.50]))
    )
].copy()

# -----
# Combine candidates
# -----

cost_simulation_df = pd.concat(

```



```

        [log_reg_cost_candidates, rf_cost_candidates],
        ignore_index=True
    )

    # Calculate total business cost
    cost_simulation_df["total_business_cost"] = cost_simulation_df.apply(
        lambda row: calculate_business_cost(
            false_positives=row["false_positives"],
            false_negatives=row["false_negatives"]
        ),
        axis=1
    )

    # Create no-model baseline:
    # If all borrowers are approved, all actual defaulters become missed defaulters
    all_approve_false_negatives = int(y_test.sum())
    all_approve_false_positives = 0
    all_approve_cost = calculate_business_cost(
        false_positives=all_approve_false_positives,
        false_negatives=all_approve_false_negatives
    )

    # Add savings compared with all-approve baseline
    cost_simulation_df["cost_saved_vs_all_approve"] = (
        all_approve_cost - cost_simulation_df["total_business_cost"]
    )

    cost_simulation_df["cost_reduction_vs_all_approve_pct"] = (
        cost_simulation_df["cost_saved_vs_all_approve"] / all_approve_cost * 100
    ).round(2)

    # Select and order columns
    cost_columns = [
        "model",
        "threshold",
        "precision",
        "recall",
        "f1_score",
        "false_positives",
        "false_negatives",
        "total_business_cost",
        "cost_saved_vs_all_approve",
        "cost_reduction_vs_all_approve_pct"
    ]

    cost_simulation_view = cost_simulation_df[cost_columns].sort_values(
        by="total_business_cost",
        ascending=True
    )

    display(cost_simulation_view.round(4))

    print("No-model baseline: approve all borrowers")

```

```

print(f"False negatives: {all_approve_false_negatives}")
print(f"False positives: {all_approve_false_positives}")
print(f"Total business cost: {all_approve_cost}")

# Identify lowest-cost model-threshold combination
best_cost_candidate = cost_simulation_view.iloc[0]

print("\nLowest-cost model-threshold combination:")
display(pd.DataFrame([best_cost_candidate]).round(4))

# Plot business cost comparison
ax = cost_simulation_view.set_index(
    cost_simulation_view["model"] + " | Threshold " + cost_simulation_view["th
")["total_business_cost"].plot(
    kind="bar",
    figsize=(10, 5)
)

ax.set_title("Business Cost Simulation by Model and Threshold")
ax.set_xlabel("Model and Threshold")
ax.set_ylabel("Total Business Cost")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()

print("Business cost simulation completed successfully.")

```

Cost assumptions:

False Negative cost: 10 cost units

False Positive cost: 1 cost unit

	model	threshold	precision	recall	f1_score	false_positives	false_negativ
2	Random Forest Baseline	0.30	0.7470	0.7954	0.7704	383	2
3	Random Forest Baseline	0.40	0.8698	0.7567	0.8093	161	3
5	Random Forest Tuned	0.35	0.8907	0.7511	0.8150	131	3
0	Logistic Regression Baseline	0.40	0.4682	0.8347	0.5999	1348	2
6	Random Forest Tuned	0.45	0.9575	0.7286	0.8275	46	3
4	Random Forest Baseline	0.50	0.9426	0.7271	0.8210	63	3
1	Logistic Regression Baseline	0.50	0.5418	0.7883	0.6422	948	3
7	Random Forest Tuned	0.50	0.9714	0.7159	0.8243	30	4

No-model baseline: approve all borrowers

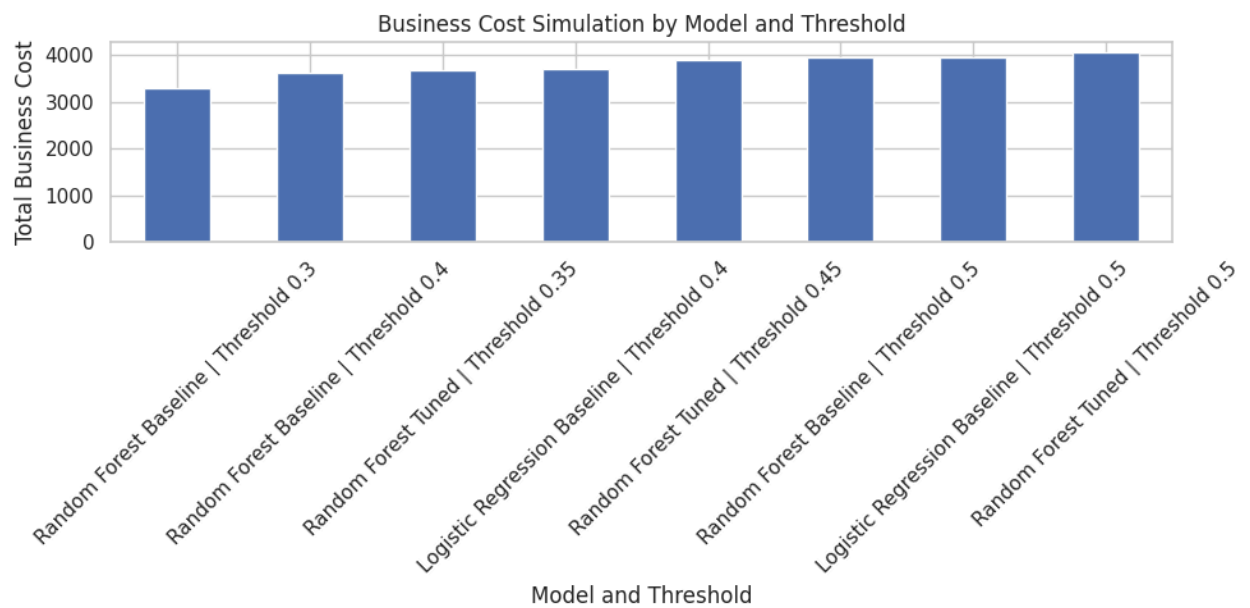
False negatives: 1422

False positives: 0

Total business cost: 14220

Lowest-cost model-threshold combination:

	model	threshold	precision	recall	f1_score	false_positives	false_negatives
2	Random Forest Baseline	0.3	0.747	0.7954	0.7704	383	291



Business cost simulation completed successfully.

6.17 Final Model Candidate Selection Based on Cost Simulation

The business cost simulation compared multiple model-threshold combinations using a simplified cost assumption:

- False Negative cost = 10 cost units
- False Positive cost = 1 cost unit

This reflects the credit-risk assumption that approving a borrower who later defaults is more costly than flagging a borrower who would have repaid.

No-Model Baseline

If all borrowers are approved, every actual defaulter becomes a missed defaulter.

In the test set, this creates:

- False negatives: 1,422
- False positives: 0
- Total business cost: 14,220

This provides a simple business baseline for evaluating whether the machine learning model creates value.

Lowest-Cost Model-Threshold Combination

The lowest-cost candidate is:

- Model: Random Forest Baseline
- Threshold: 0.30
- Precision: 0.7470
- Recall: 0.7954
- F1-score: 0.7704
- False positives: 383
- False negatives: 291
- Total business cost: 3,293
- Cost reduction vs all-approve baseline: 76.84%

This model-threshold combination substantially reduces missed defaulters while keeping false positives at a manageable level.

Why Random Forest Baseline at Threshold 0.30 Is Selected

Although the tuned Random Forest achieved a slightly higher F1-score at threshold 0.45, the cost simulation shows that it is not the best business option under the selected cost assumptions.

The tuned Random Forest is more conservative and produces fewer false positives, but it misses more actual defaulters. Because false negatives are assumed to be much more expensive, the Random Forest baseline at threshold 0.30 provides better business value.

This demonstrates an important machine learning principle:

■ The statistically best model is not always the best business model.

For this project, the final recommended model candidate is the Random Forest baseline with a decision threshold of 0.30.

Recommended Decision-Support Use

The model should not be used as an automatic rejection system. Instead, it should support a risk-based lending workflow:

- Low predicted default probability: standard approval review
- Medium predicted default probability: manual credit-risk review
- High predicted default probability: enhanced underwriting or additional

documentation

This approach balances risk reduction with responsible decision-making and avoids fully automated denial of credit access.

```
In [24]: # =====
# Final Selected Model Evaluation
# =====

# Final model selected based on business cost simulation
final_model = rf_model
final_model_name = "Final Selected Model: Random Forest Baseline"
final_threshold = 0.30

# Evaluate final selected model
final_model_metrics = evaluate_classification_model(
    model=final_model,
    X_data=X_test,
    y_true=y_test,
    model_name=final_model_name,
    threshold=final_threshold
)

# Add business-cost results to final metrics
final_model_metrics["false_negative_cost"] = COST_FALSE_NEGATIVE
final_model_metrics["false_positive_cost"] = COST_FALSE_POSITIVE
final_model_metrics["total_business_cost"] = calculate_business_cost(
    false_positives=final_model_metrics["false_positives"],
    false_negatives=final_model_metrics["false_negatives"]
)

final_model_summary = pd.DataFrame([final_model_metrics])

display(final_model_summary.round(4))

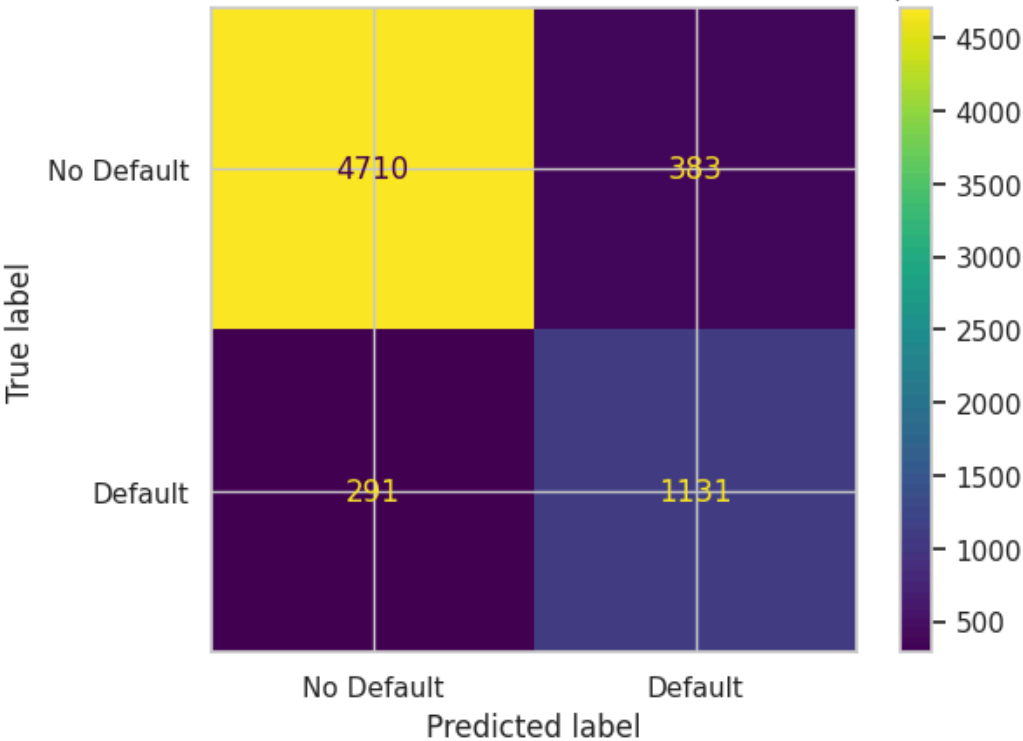
print("Final selected model evaluation completed successfully.")
print(f"Selected model: {final_model_name}")
print(f"Selected threshold: {final_threshold}")
print(f"Total business cost: {final_model_metrics['total_business_cost']}")
```

	model	threshold	accuracy	precision	recall	f1_score	roc_auc	pr_auc
0	Final Selected Model: Random Forest Baseline	0.3	0.896546	0.747028	0.795359	0.770436	0.930451	0.88103

Classification Report: Final Selected Model: Random Forest Baseline

	precision	recall	f1-score	support
No Default	0.94	0.92	0.93	5093
Default	0.75	0.80	0.77	1422
accuracy			0.90	6515
macro avg	0.84	0.86	0.85	6515
weighted avg	0.90	0.90	0.90	6515

Confusion Matrix — Final Selected Model: Random Forest Baseline | Threshold = 0.3



	model	threshold	accuracy	precision	recall	f1_score	roc_auc	pr_auc	true
0	Final Selected Model: Random Forest Baseline	0.3	0.8965	0.747	0.7954	0.7704	0.9305	0.881	

Final selected model evaluation completed successfully.
Selected model: Final Selected Model: Random Forest Baseline
Selected threshold: 0.3
Total business cost: 3293

7. Model Interpretation

7.1 Objective

After selecting the final model, it is important to understand which features contribute most to the model's predictions.

The final selected model is:

- Model: Random Forest Baseline
- Decision threshold: 0.30

The model should be interpreted as a risk-ranking and decision-support tool. The selected threshold is applied to predicted default probabilities, but feature importance comes from the trained Random Forest model itself.

This section analyzes feature importance to understand the main drivers of predicted default risk.

The interpretation focuses on:

- Which features are most influential
- Whether the model learns patterns that make business sense
- How model outputs can support credit-risk decision-making
- Limitations of feature importance in a real-world lending context

Because the model includes one-hot encoded categorical variables, feature importance is reviewed at two levels:

1. Encoded feature level
2. Aggregated original feature level

```
In [25]: # =====  
# Model Interpretation: Feature Importance  
# =====  
  
# Extract fitted preprocessing and model steps from the final selected model  
final_preprocessor = final_model.named_steps["preprocessor"]  
final_rf_classifier = final_model.named_steps["model"]  
  
# Extract encoded categorical feature names  
encoded_categorical_features = (  
    final_preprocessor  
    .named_transformers_["cat"]  
    .get_feature_names_out(categorical_cols)  
)
```



```

# Combine numerical and encoded categorical feature names
encoded_feature_names = numerical_cols + list(encoded_categorical_features)

# Extract Random Forest feature importances
feature_importance_df = pd.DataFrame({
    "encoded_feature": encoded_feature_names,
    "importance": final_rf_classifier.feature_importances_
})

feature_importance_df = feature_importance_df.sort_values(
    by="importance",
    ascending=False
).reset_index(drop=True)

# Display top encoded features
top_encoded_features = feature_importance_df.head(15)
display(top_encoded_features)

# Plot top encoded feature importances
plt.figure(figsize=(9, 6))
sns.barplot(
    data=top_encoded_features,
    x="importance",
    y="encoded_feature"
)

plt.title("Top 15 Encoded Feature Importances – Final Random Forest Model")
plt.xlabel("Importance")
plt.ylabel("Encoded Feature")
plt.tight_layout()
plt.show()

# =====
# Aggregate encoded feature importances back to original features
# =====

def map_to_original_feature(encoded_feature, original_features):
    """
    Map an encoded feature name back to its original feature.
    """
    for original_feature in original_features:
        if encoded_feature == original_feature:
            return original_feature
        if encoded_feature.startswith(original_feature + "_"):
            return original_feature
    return encoded_feature

original_feature_list = numerical_cols + categorical_cols

feature_importance_df["original_feature"] = feature_importance_df["encoded_fea
    lambda x: map_to_original_feature(x, original_feature_list)

```

```
)

aggregated_feature_importance = (
    feature_importance_df
    .groupby("original_feature", as_index=False)["importance"]
    .sum()
    .sort_values(by="importance", ascending=False)
    .reset_index(drop=True)
)

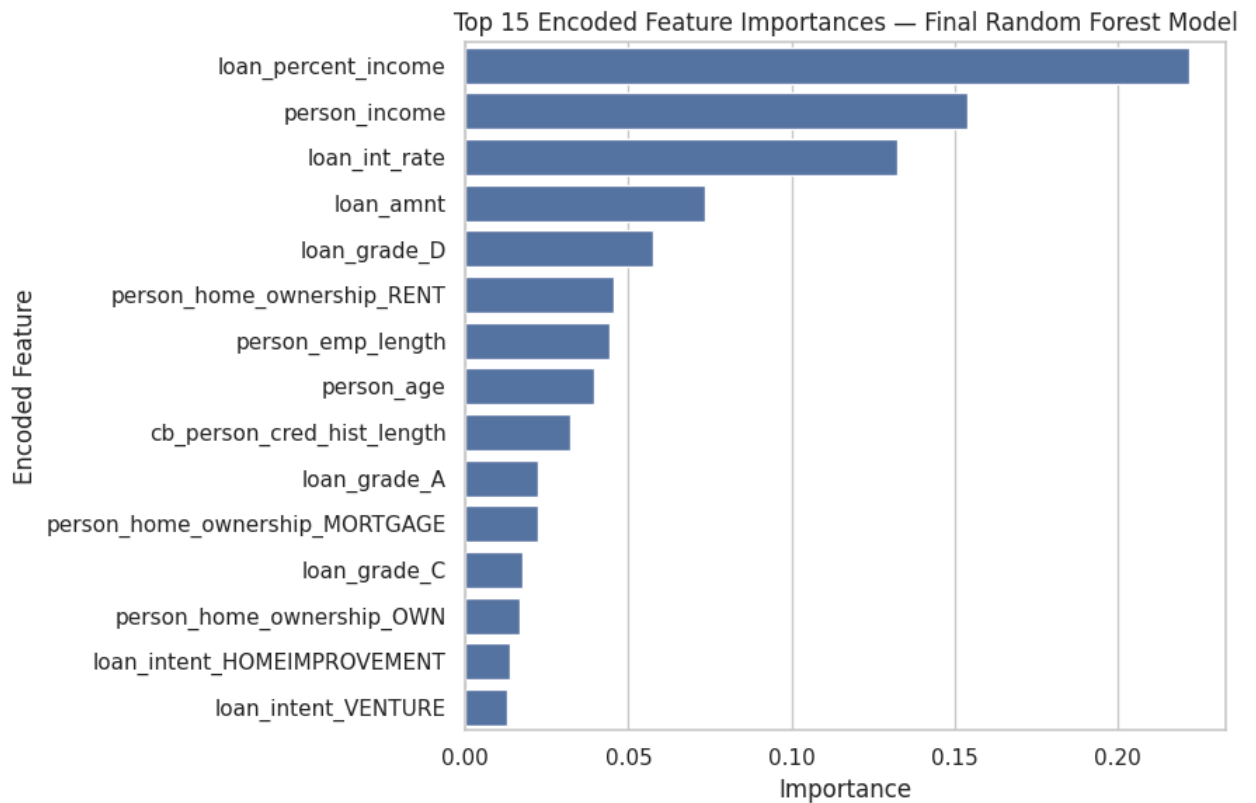
display(aggregated_feature_importance)

# Plot aggregated feature importances
plt.figure(figsize=(9, 6))
sns.barplot(
    data=aggregated_feature_importance,
    x="importance",
    y="original_feature"
)

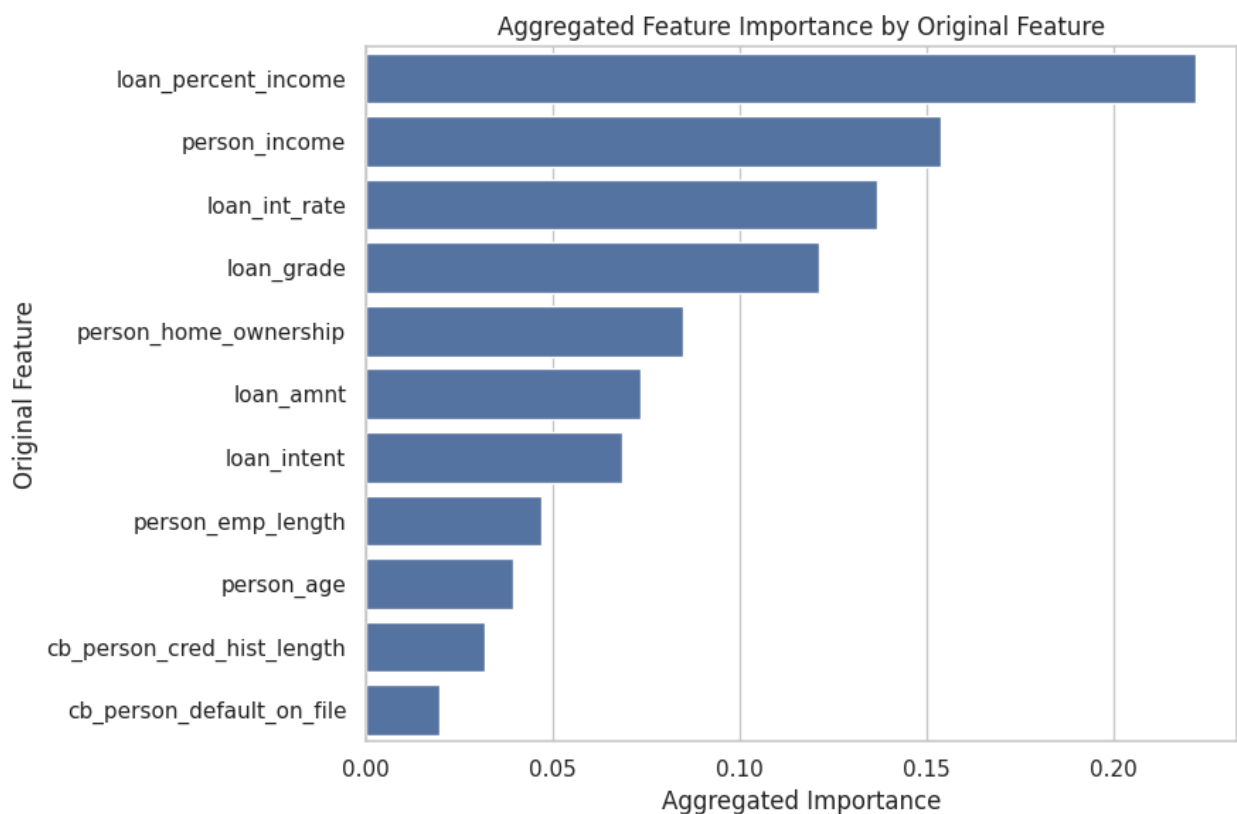
plt.title("Aggregated Feature Importance by Original Feature")
plt.xlabel("Aggregated Importance")
plt.ylabel("Original Feature")
plt.tight_layout()
plt.show()

print("Model interpretation completed successfully.")
```

	encoded_feature	importance
0	loan_percent_income	0.221890
1	person_income	0.153940
2	loan_int_rate	0.132416
3	loan_amnt	0.073601
4	loan_grade_D	0.057385
5	person_home_ownership_RENT	0.045655
6	person_emp_length	0.044218
7	person_age	0.039521
8	cb_person_cred_hist_length	0.032100
9	loan_grade_A	0.022429
10	person_home_ownership_MORTGAGE	0.022273
11	loan_grade_C	0.017458
12	person_home_ownership_OWN	0.016699
13	loan_intent_HOMEIMPROVEMENT	0.013782
14	loan_intent_VENTURE	0.012924



	original_feature	importance
0	loan_percent_income	0.221890
1	person_income	0.153940
2	loan_int_rate	0.136677
3	loan_grade	0.121350
4	person_home_ownership	0.085155
5	loan_amnt	0.073601
6	loan_intent	0.068839
7	person_emp_length	0.047091
8	person_age	0.039521
9	cb_person_cred_hist_length	0.032100
10	cb_person_default_on_file	0.019836



Model interpretation completed successfully.

7.2 Business Interpretation of Feature Importance

The feature-importance analysis shows that the final Random Forest model is mainly driven by borrower affordability, income, risk pricing, loan grade, and

housing profile.

This aligns well with credit-risk logic and supports the credibility of the model.

1. Loan Percent Income Is the Strongest Risk Driver

The most important feature is `loan_percent_income`.

This variable measures the loan amount relative to borrower income. A higher value means that the loan represents a larger share of the borrower's income.

From a credit-risk perspective, this is a strong affordability signal. Borrowers with a high loan-to-income ratio may have less repayment capacity and may be more vulnerable to default.

This confirms one of the main EDA findings: defaulted borrowers had a higher median loan percent income than non-defaulted borrowers.

2. Borrower Income Is Highly Important

The second most important feature is `person_income`.

Income is directly related to repayment capacity. Borrowers with lower income may have less financial flexibility to manage loan repayments, especially when loan amounts or interest rates are high.

The model's use of income is consistent with the business interpretation that borrower financial capacity is central to credit-risk assessment.

3. Interest Rate Reflects Risk Pricing

`loan_int_rate` is also one of the most important features.

Higher interest rates often reflect higher perceived borrower risk. In the EDA, defaulted borrowers had a higher median interest rate than non-defaulted borrowers.

This suggests that interest rate contains useful information about risk. However, it should be interpreted carefully because it may partially reflect previous lender risk-assessment decisions.

4. Loan Grade Is an Important Institutional Risk Signal

`loan_grade` is another important predictor.

The categorical EDA showed a strong risk gradient across loan grades. Lower-quality grades had substantially higher default rates than higher-quality grades.

This confirms that the model is learning meaningful institutional credit-risk patterns.

Loan grade should be interpreted as a strong summary signal, but in a real-world lending system it would also be important to understand how the grade was originally assigned.

5. Home Ownership Adds Borrower Profile Information

`person_home_ownership` is also among the most important original features.

The EDA showed that renters had a higher default rate than borrowers who owned a home or had a mortgage. This may reflect differences in financial stability, asset ownership, or borrower profile.

However, this feature requires careful responsible AI consideration. Housing status may be correlated with socioeconomic factors, so it should not be used without governance, fairness review, and human oversight in a real lending system.

6. Interpretation Limitations

Feature importance helps explain which variables the model uses most, but it does not prove causality.

Important limitations include:

- Random Forest feature importance does not show whether a feature increases or decreases default risk.
- Correlated variables can share importance.
- Features with many possible split points may receive higher importance.
- Business interpretation must be combined with EDA, threshold analysis, and responsible AI review.

Therefore, the feature-importance results should be used to support model understanding, not as the only explanation for individual loan decisions.

7. Summary

The final model's most important features are consistent with credit-risk logic:

- Loan affordability
- Borrower income
- Interest rate
- Loan grade
- Home ownership profile
- Loan amount and loan purpose

This strengthens confidence that the model is learning meaningful credit-risk patterns rather than random noise.

8. Responsible AI and Fairness Considerations

8.1 Objective

Credit-risk modeling is a high-impact machine learning use case because model predictions may influence access to financial products.

For that reason, this project should not only evaluate predictive performance. It should also consider responsible AI, fairness, transparency, and governance.

The final model is designed as a decision-support tool, not as a fully automated loan rejection system.

8.2 Why Responsible AI Matters in Credit Risk

A credit-risk model can affect real people by influencing decisions such as:

- Loan approval
- Manual review priority
- Risk-based lending strategy
- Additional documentation requests
- Credit access opportunities

Incorrect or poorly governed model use may create harmful outcomes, especially if the model learns patterns that disadvantage certain borrower groups.

Therefore, responsible use requires:

- Human oversight
 - Clear documentation
 - Careful threshold selection
 - Monitoring model performance over time
 - Reviewing subgroup performance
 - Avoiding fully automated rejection decisions without governance
-

8.3 Dataset Limitations

This dataset does not include explicit protected attributes such as race, ethnicity, religion, disability status, or gender.

Because these variables are not available, a full fairness audit cannot be performed.

However, the absence of protected attributes does not automatically mean the model is fair. Some available variables may act as proxies for socioeconomic status or other sensitive factors.

Examples include:

- `person_income`
- `person_home_ownership`
- `loan_grade`
- `loan_int_rate`

These variables may be important for credit-risk prediction, but they should be reviewed carefully in a real-world lending system.

8.4 Potential Fairness Risks

Several fairness risks should be considered:

1. **Proxy discrimination**

- Variables such as income, housing status, and loan grade may correlate with socioeconomic disadvantage.

2. **Historical bias**

- If historical lending decisions were biased, the model may learn and reproduce those patterns.

3. **Unequal error rates**

- The model may perform better for some borrower groups than others.

4. **Threshold impact**

- A selected threshold may reduce business cost overall but still affect groups differently.

5. **Lack of individual explanation**

- Random Forest feature importance helps explain global model behavior, but it does not fully explain individual decisions.

8.5 Responsible Use Recommendation

The model should be used as part of a human-in-the-loop credit-risk workflow.

Recommended use:

- Use predicted default probability as a risk score.
- Use the model to prioritize applications for manual review.
- Avoid automatic rejection based only on model output.
- Review high-risk predictions with additional borrower information.
- Monitor model performance after deployment.
- Reassess fairness and subgroup error rates regularly.
- Document model assumptions, limitations, and threshold choices.

This approach supports risk reduction while maintaining responsible decision-making.

```
In [26]: # =====
# Responsible AI Diagnostic: Subgroup Performance
# =====

# Create test-set predictions for the final selected model
final_test_proba = final_model.predict_proba(X_test)[: , 1]
final_test_pred = (final_test_proba >= final_threshold).astype(int)

# Combine test features, true labels, predictions, and probabilities
responsible_ai_test_df = X_test.copy()
responsible_ai_test_df["y_true"] = y_test.values
responsible_ai_test_df["y_pred"] = final_test_pred
responsible_ai_test_df["default_probability"] = final_test_proba

def subgroup_performance_table(data, group_col):
```

```

"""
Calculate subgroup-level performance metrics for a categorical feature.
"""
subgroup_results = []

for group_value, group_data in data.groupby(group_col):
    y_true_group = group_data["y_true"]
    y_pred_group = group_data["y_pred"]

    # Skip very small groups to avoid unstable metrics
    if len(group_data) < 30:
        continue

    # Use labels=[0, 1] to make the confusion matrix robust
    tn, fp, fn, tp = confusion_matrix(
        y_true_group,
        y_pred_group,
        labels=[0, 1]
    ).ravel()

    subgroup_results.append({
        "group_feature": group_col,
        "group_value": group_value,
        "group_size": len(group_data),
        "actual_default_rate": y_true_group.mean(),
        "predicted_default_rate": y_pred_group.mean(),
        "precision": precision_score(y_true_group, y_pred_group, zero_divi
        "recall": recall_score(y_true_group, y_pred_group, zero_division=0)
        "f1_score": f1_score(y_true_group, y_pred_group, zero_division=0),
        "false_positives": fp,
        "false_negatives": fn,
        "true_positives": tp,
        "true_negatives": tn
    })

return pd.DataFrame(subgroup_results)

# Subgroup checks for available categorical variables
subgroup_features = [
    "person_home_ownership",
    "loan_intent",
    "loan_grade",
    "cb_person_default_on_file"
]

subgroup_performance_results = []

for feature in subgroup_features:
    subgroup_table = subgroup_performance_table(
        responsible_ai_test_df,
        group_col=feature
    )

```

```

subgroup_performance_results.append(subgroup_table)

subgroup_performance_df = pd.concat(
    subgroup_performance_results,
    ignore_index=True
)

# Format selected rate columns
subgroup_display = subgroup_performance_df.copy()
subgroup_display["actual_default_rate"] = (
    subgroup_display["actual_default_rate"] * 100
).round(2)

subgroup_display["predicted_default_rate"] = (
    subgroup_display["predicted_default_rate"] * 100
).round(2)

display(subgroup_display.round(4))

print("Responsible AI subgroup diagnostic completed successfully.")
print("Important: This is not a full fairness audit because protected attributes")
print("The table should be used to identify groups where model performance may

```

	group_feature	group_value	group_size	actual_default_rate	p
0	person_home_ownership	MORTGAGE	2673	13.69	
1	person_home_ownership	OWN	525	7.62	
2	person_home_ownership	RENT	3291	30.75	
3	loan_intent	DEBTCONSOLIDATION	1075	30.79	
4	loan_intent	EDUCATION	1296	15.97	
5	loan_intent	HOMEIMPROVEMENT	751	25.70	
6	loan_intent	MEDICAL	1177	26.25	
7	loan_intent	PERSONAL	1077	19.87	
8	loan_intent	VENTURE	1139	14.75	
9	loan_grade	A	2131	9.81	
10	loan_grade	B	2063	16.04	
11	loan_grade	C	1307	20.58	
12	loan_grade	D	751	57.92	
13	loan_grade	E	193	65.80	
14	loan_grade	F	50	62.00	
15	cb_person_default_on_file	N	5332	18.53	
16	cb_person_default_on_file	Y	1183	36.69	

Responsible AI subgroup diagnostic completed successfully.

Important: This is not a full fairness audit because protected attributes are not available.

The table should be used to identify groups where model performance may require closer review.

8.6 Responsible AI Diagnostic Interpretation

The subgroup diagnostic provides an initial review of model performance across available categorical groups.

This is not a full fairness audit because the dataset does not contain protected attributes such as gender, ethnicity, disability status, or religion. However, subgroup analysis is still useful because it can identify areas where model performance differs across borrower segments.

1. Home Ownership Groups

Model performance differs across home ownership categories.

For borrowers with **OWN** home ownership, recall is very high, meaning the model detects default cases effectively in this group.

For borrowers with **RENT**, the model also performs strongly, with high recall and good F1-score.

However, for borrowers with **MORTGAGE**, recall is lower. This means the model misses more default cases in this group compared with some other home ownership categories.

This difference should be reviewed carefully in a real lending context. Housing status may be related to borrower financial stability, but it may also correlate with socioeconomic factors. Therefore, this feature should be monitored from a responsible AI perspective.

2. Loan Intent Groups

The model shows different performance across loan purposes.

Some loan purposes, such as **DEBTCONSOLIDATION**, **MEDICAL**, and **VENTURE**, show relatively strong recall and F1-score. Other groups, such as **EDUCATION** and **PERSONAL**, show weaker performance.

This suggests that the model may identify risk more effectively for some loan

purposes than others.

In a real credit-risk workflow, loan intent should be monitored to ensure that model errors do not disproportionately affect specific borrower segments.

3. Loan Grade Groups

The model performs very strongly for higher-risk loan grades such as **D**, **E**, and **F**, with very high recall.

This means the model is effective at identifying defaults among borrowers already assigned weaker credit grades.

However, for lower-risk grades such as **A**, **B**, and **C**, recall is lower. This indicates that some defaulters in lower-risk grades are harder for the model to detect.

This is an important operational insight: borrowers with apparently safer loan grades may still default, and the model should not rely only on grade-based risk patterns.

4. Previous Default History

Borrowers with previous default history show a higher actual default rate and higher predicted default rate.

The model also achieves strong recall for this group, suggesting that it captures previous default history as an important risk signal.

However, previous default history should be used carefully. In real lending systems, historical credit behavior may reflect structural or economic disadvantages.

Therefore, this feature should support risk review but should not be the only basis for adverse decisions.

5. Responsible AI Summary

The subgroup diagnostic shows that the model has useful predictive power, but performance varies across borrower groups.

Key responsible AI conclusions:

- The model should not be used for fully automated rejection.
- High-risk predictions should be reviewed by a human decision-maker.

- Subgroup performance should be monitored regularly.
- Thresholds should be reviewed for both business impact and fairness impact.
- Additional fairness analysis would be required before any real-world deployment.
- The absence of protected attributes does not guarantee that the model is fair.

This project therefore positions the model as a decision-support system rather than a fully automated credit decision engine.

9. Final Recommendations and Conclusion

9.1 Final Model Recommendation

After comparing multiple models, thresholds, and business-cost scenarios, the final recommended model is:

- **Model:** Random Forest Baseline
- **Decision threshold:** 0.30
- **Use case:** Credit-risk decision support and manual review prioritization

The final model was selected because it provided the best business value under the assumed cost structure.

9.2 Final Model Performance

The final selected model achieved the following test-set performance:

Metric	Result
Accuracy	0.8985
Precision	0.7470
Recall	0.7954
F1-score	0.7704
ROC-AUC	0.9305
PR-AUC	0.8810
False Positives	383
False Negatives	291

Metric	Result
True Positives	1,131
True Negatives	4,710

The selected threshold increases default detection compared with the default 0.50 threshold. This is important because missed defaulters are assumed to be more costly than false risk flags.

9.3 Business Cost Simulation Result

The cost simulation used the following simplified assumptions:

Error Type	Assumed Cost
False Negative	10 cost units
False Positive	1 cost unit

The no-model baseline assumes that all borrowers are approved. Under this scenario, all actual defaulters become missed defaulters.

Scenario	Total Business Cost
Approve all borrowers	14,220
Final selected model	3,293

The final selected model reduces estimated business cost by:

- **10,927 cost units**
- **76.84% cost reduction compared with approving all borrowers**

This demonstrates that the model provides meaningful business value under the selected cost assumptions.

9.4 Recommended Business Use

The model should be used as a **decision-support tool**, not as an automatic loan rejection system.

A practical credit-risk workflow could use predicted default probability bands:

Risk Band	Suggested Action
Low predicted default probability	Standard approval review

Risk Band	Suggested Action
Medium predicted default probability	Manual credit-risk review
High predicted default probability	Enhanced underwriting or additional documentation

This approach allows the lender to reduce default risk while still maintaining human oversight.

9.5 Key Risk Drivers

The model interpretation showed that the most important risk drivers are:

- `loan_percent_income`
- `person_income`
- `loan_int_rate`
- `loan_grade`
- `person_home_ownership`
- `loan_amnt`
- `loan_intent`

These features are consistent with credit-risk logic because they reflect borrower affordability, repayment capacity, institutional risk pricing, and loan characteristics.

The strongest feature was `loan_percent_income`, confirming that borrower affordability is a central driver of default risk.

9.6 Responsible AI Recommendation

Credit-risk modeling is a high-impact use case because model outputs may influence access to financial services.

Therefore, the final model should be used responsibly.

Recommended responsible AI safeguards include:

- Keep a human-in-the-loop review process.
- Avoid fully automated rejection decisions.
- Monitor subgroup performance over time.
- Review threshold impact across borrower segments.
- Document model assumptions and limitations.
- Reassess model performance after deployment.

- Perform a full fairness audit before any real-world use.

The subgroup diagnostic showed that model performance varies across available borrower groups, so ongoing monitoring is necessary.

9.7 Limitations

This project has several limitations:

1. **Dataset limitations**

- The dataset is publicly available and may not represent the full complexity of real lending data.

2. **No protected attributes**

- The dataset does not include protected demographic attributes, so a full fairness audit cannot be performed.

3. **Simplified cost simulation**

- The business cost assumptions are hypothetical. Real-world costs would depend on loan amount, recovery rate, interest income, and operational review cost.

4. **Global feature importance**

- Random Forest feature importance explains overall model behavior but does not provide complete individual-level explanations.

5. **No production monitoring**

- This notebook does not include live deployment, drift monitoring, or model governance infrastructure.
-

9.8 Future Improvements

Future improvements could include:

- Testing additional models such as XGBoost or LightGBM.
- Using SHAP values for stronger model explainability.
- Building a probability-band decision framework.
- Running sensitivity analysis with different false-positive and false-negative cost ratios.
- Adding calibration analysis to check whether predicted probabilities are

reliable.

- Performing a deeper fairness audit if protected or proxy-sensitive variables are available.
 - Creating a dashboard for credit-risk monitoring and business reporting.
-

9.9 Final Conclusion

This project demonstrates an end-to-end machine learning workflow for credit-risk analysis and loan default prediction.

The final solution combines:

- Business problem framing
- Data quality management
- Exploratory data analysis
- Feature engineering
- Logistic Regression baseline modeling
- Random Forest modeling
- Cross-validation
- Hyperparameter tuning
- Threshold analysis
- Business cost simulation
- Feature importance interpretation
- Responsible AI considerations

The final selected model, Random Forest Baseline with a threshold of 0.30, provides the best business value under the assumed cost structure. It reduces estimated business cost substantially compared with approving all borrowers, while maintaining strong predictive performance.

The project shows that effective credit-risk modeling requires more than high accuracy. A useful model must balance predictive performance, business impact, interpretability, and responsible decision-making.